

**UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE**

**TECNOLOGÍAS DE LA INFORMACIÓN**



**EXAMEN SEGUNDO PARCIAL**

**NRC: 30060**

**MATERIA:** Inteligencia Artificial

**DOCENTE:** Ing. Mauricio Loachamin

**ESTUDIANTES:** Nicole Carvajal y César Gonzalez

**FECHA:** 22/12/2025

**PREGRADO SII OCT 2025 – FEB 2026**

## Introducción

En el presente informe se documenta el proceso de implementación de un modelo de aprendizaje automático para el reconocimiento de imágenes de perros y gatos, así como la verificación de su correcto funcionamiento mediante un aplicativo web. El trabajo se basa en el uso de técnicas de aprendizaje supervisado, empleando Python y la librería TensorFlow para el entrenamiento y evaluación del modelo.

Inicialmente, se describe paso a paso el desarrollo del proyecto, comenzando con la obtención y análisis del conjunto de datos, el preprocesamiento de las imágenes y la definición de distintas arquitecturas de redes neuronales. Posteriormente, se realiza el entrenamiento del modelo, se evalúa su desempeño utilizando métricas de precisión y pérdida, y se analizan los resultados obtenidos para identificar posibles problemas como el sobreajuste. Finalmente, el modelo entrenado es exportado e integrado en una aplicación web que permite realizar predicciones en tiempo real utilizando la cámara del dispositivo.

Como segunda fase del proyecto, se plantea la ampliación del modelo para el reconocimiento de un tercer tipo de animal, utilizando un nuevo conjunto de datos. En esta etapa, se entrena nuevamente el modelo incorporando la nueva clase, con el objetivo de analizar su capacidad de generalización y adaptación ante un problema de clasificación más complejo. De esta manera, el informe presenta una visión completa del desarrollo, implementación y extensión de un sistema de clasificación de imágenes basado en redes neuronales convolucionales.

## Objetivos

### Objetivo General

Implementar y evaluar un modelo de aprendizaje automático capaz de reconocer imágenes de perros y gatos, verificando su funcionamiento mediante un aplicativo web, y posteriormente extender dicho modelo para el reconocimiento de un tercer tipo de animal utilizando un nuevo conjunto de datos.

### Objetivos Específicos

- Analizar y preparar un conjunto de datos de imágenes de perros y gatos para su uso en un modelo de aprendizaje supervisado.
- Aplicar técnicas de preprocesamiento de imágenes, como redimensionamiento, normalización y conversión a escala de grises, con el fin de optimizar el entrenamiento del modelo.
- Diseñar y entrenar diferentes arquitecturas de redes neuronales, incluyendo redes densas y redes neuronales convolucionales.
- Evaluar el desempeño de los modelos entrenados mediante métricas de precisión y pérdida, identificando posibles problemas como el sobreajuste.
- Implementar técnicas de aumento de datos para mejorar la capacidad de generalización del modelo.
- Exportar el modelo entrenado e integrarlo en una aplicación web que permita realizar predicciones en tiempo real utilizando la cámara del dispositivo.
- Entrenar nuevamente el modelo incorporando un tercer tipo de animal mediante un nuevo conjunto de datos.
- Analizar el impacto de la inclusión de una nueva clase en el rendimiento y comportamiento del modelo..

## **Marco Teórico**

### **Aprendizaje automático**

El aprendizaje automático (Machine Learning) es una rama de la inteligencia artificial que permite a los sistemas aprender a partir de datos sin necesidad de ser programados explícitamente para cada tarea. Su objetivo principal es identificar patrones en grandes volúmenes de información y utilizarlos para realizar predicciones o clasificaciones. En este proyecto se emplea el aprendizaje supervisado, donde el modelo es entrenado con datos previamente etiquetados, permitiéndole asociar características de entrada con una salida conocida. (*¿Qué Es El Aprendizaje Automático? Tipos Y Usos*, 2025)

### **Aprendizaje supervisado**

El aprendizaje supervisado se basa en el uso de conjuntos de datos que contienen ejemplos de entrada junto con sus respectivas etiquetas. Durante el entrenamiento, el modelo ajusta sus parámetros internos para minimizar el error entre las predicciones realizadas y los valores reales. En el contexto de este informe, las imágenes de animales se encuentran etiquetadas según su clase (perro, gato y posteriormente un tercer animal), lo que permite entrenar un modelo de clasificación de imágenes. (Belcic & Stryker, 2025)

### **Redes neuronales artificiales**

Las redes neuronales artificiales son modelos computacionales inspirados en el funcionamiento del cerebro humano. Están compuestas por capas de neuronas interconectadas que procesan la información mediante operaciones matemáticas. Cada neurona recibe entradas, aplica una función de activación y produce una salida que se



transmite a la siguiente capa. Estas redes son ampliamente utilizadas en problemas de clasificación debido a su capacidad para modelar relaciones complejas entre los datos. (Alicante, 2025)

### **Redes neuronales convolucionales (CNN)**

Las redes neuronales convolucionales (Convolutional Neural Networks) son un tipo especializado de red neuronal diseñado para el procesamiento de imágenes. Utilizan capas de convolución que permiten extraer características relevantes como bordes, formas y texturas. Estas redes incluyen, además, capas de agrupamiento (pooling) que reducen la dimensionalidad de los datos y capas densas que realizan la clasificación final. Las CNN son especialmente efectivas en tareas de reconocimiento de imágenes, como la clasificación de animales.

### **Preprocesamiento de imágenes**

El preprocesamiento de imágenes consiste en aplicar transformaciones previas a los datos antes del entrenamiento del modelo. Entre las técnicas más utilizadas se encuentran el redimensionamiento de imágenes, la normalización de valores de píxeles y la conversión a escala de grises. Estas técnicas permiten estandarizar los datos de entrada, reducir la complejidad computacional y mejorar el rendimiento del modelo. (IBM, 2021)

### **Aumento de datos**

El aumento de datos (Data Augmentation) es una técnica utilizada para incrementar artificialmente la cantidad de datos de entrenamiento mediante transformaciones como rotaciones, desplazamientos, volteos y escalados de las imágenes. Esta estrategia ayuda a

reducir el sobreajuste y mejora la capacidad de generalización del modelo, permitiéndole adaptarse mejor a datos nuevos no vistos durante el entrenamiento.

## **Desarrollo**

### **Parte 1: Perros y Gatos**

#### **1. Configuración del entorno de trabajo**

El desarrollo del proyecto se realiza en Google Colab, una plataforma en la nube que permite programar en Python sin necesidad de realizar instalaciones locales. Una de las principales ventajas de este entorno es la posibilidad de utilizar aceleración por GPU, lo cual reduce significativamente el tiempo de entrenamiento de los modelos de aprendizaje automático.

Antes de iniciar el desarrollo, se verifica que el tipo de entorno de ejecución esté configurado para utilizar GPU, ya que el entrenamiento de redes neuronales convolucionales con grandes volúmenes de imágenes puede resultar lento si se ejecuta únicamente en CPU. Esta configuración es un paso fundamental para optimizar el rendimiento del proceso de entrenamiento.

### Cambiar tipo de entorno de ejecución

Tipo de entorno de ejecución

Python 3 ▼

Acelerador por hardware ?

☐ CPU ☒ GPU T4 ☐ GPU H100 ☐ GPU A100

☐ GPU L4 ☐ TPU v5e-1 ☐ TPU v6e-1

¿Quieres acceder a GPUs premium?

[Compra unidades de computación adicionales](#)

Versión del entorno de ejecución ?

Última (recomendada) ▼

Cancelar Guardar

## Importación de librerías y obtención del conjunto de datos

Una vez configurado el entorno, se importan las librerías necesarias, principalmente TensorFlow y TensorFlow Datasets, las cuales permiten descargar, manipular y entrenar modelos de aprendizaje automático de forma eficiente.

Se utiliza un conjunto de datos previamente etiquetado que contiene imágenes de perros y gatos, el cual cuenta con aproximadamente 23 000 imágenes. Cada imagen está asociada a una etiqueta que indica la clase correspondiente, donde el valor 0 representa a un gato y el valor 1 representa a un perro. Este conjunto de datos es adecuado para aplicar aprendizaje supervisado, ya que las clases se encuentran claramente definidas.

```
import tensorflow as tf
import tensorflow_datasets as tfds

#Descargar el set de datos de perros y gatos
datos, metadatos = tfds.load('cats_vs_dogs', as_supervised=True, with_info=True)
```

... WARNING:absl:Variant folder /root/tensorflow\_datasets/cats\_vs\_dogs/4.0.1 has no dataset\_info.json  
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size) to /root/tensorflow\_dat  
DI Completed...: 100% 1/1 [00:09<00:00, 9.14s/ url]  
DI Size...: 100% 786/786 [00:09<00:00, 90.77 MiB/s]  
WARNING:absl:1738 images were corrupted and were skipped  
Dataset cats\_vs\_dogs downloaded and prepared to /root/tensorflow\_datasets/cats\_vs\_dogs/4.0.1. Subsequent calls will reuse this data.

## Exploración y visualización de los datos

Antes de entrenar el modelo, se realiza una exploración del conjunto de datos con el objetivo de comprender su estructura y características. Se visualizan varias imágenes para confirmar que los datos se hayan cargado correctamente y para observar la diversidad del contenido.

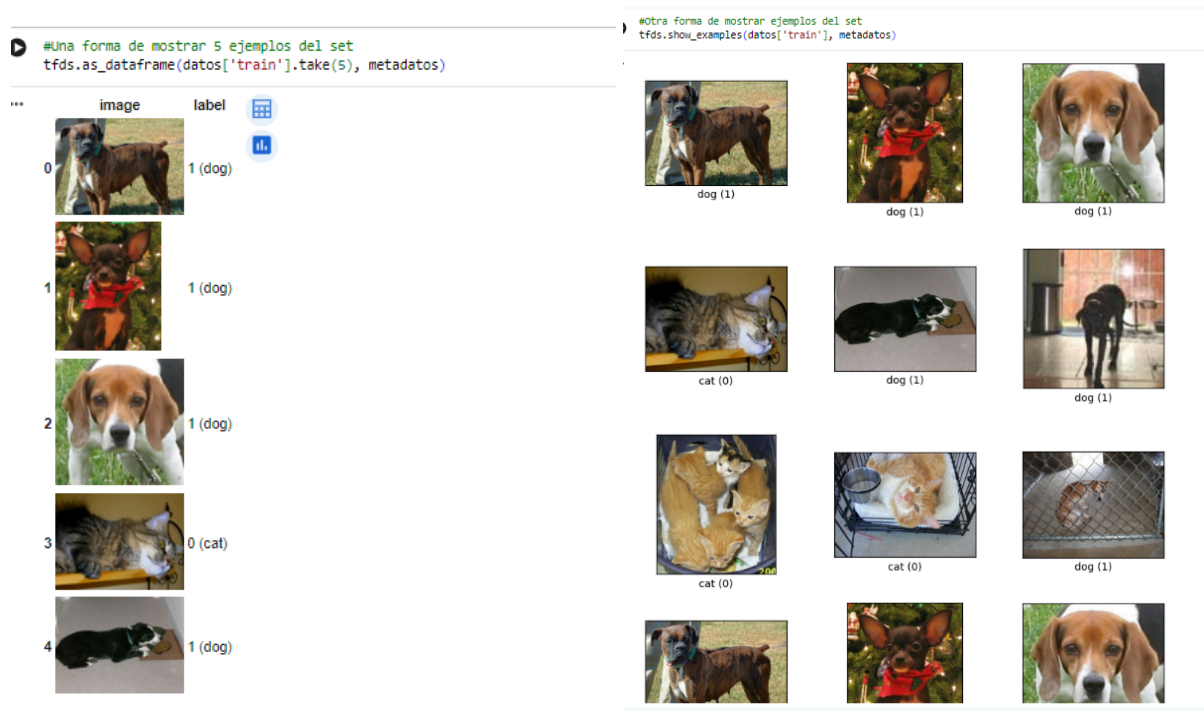
En esta etapa se realizó la inspección de los metadatos del conjunto de datos cats\_vs\_dogs mediante TensorFlow Datasets. Este análisis permitió identificar que el conjunto está compuesto por 23,262 imágenes etiquetadas de perros y gatos, correspondientes a un problema de clasificación binaria supervisada. Las imágenes se encuentran en formato RGB con valores de píxeles entre 0 y 255, y aquellas que presentaban errores fueron descartadas automáticamente.

La revisión de los metadatos facilitó la comprensión de la estructura, tamaño y características del conjunto de datos, lo cual resulta fundamental para definir el preprocesamiento y la arquitectura del modelo de aprendizaje automático a emplear. Por esta razón, se determina la necesidad de aplicar un proceso de preprocesamiento antes de utilizar las imágenes como datos de entrada.

```
#Imprimir los metadatos para revisarlos
metadatos

... tfds.core.DatasetInfo(
    name='cats_vs_dogs',
    full_name='cats_vs_dogs/4.0.1',
    description="""
A large set of images of cats and dogs. There are 1738 corrupted images that are dropped.
""",
    homepage='https://www.microsoft.com/en-us/download/details.aspx?id=54765',
    data_dir='/root/tensorflow_datasets/cats_vs_dogs/4.0.1',
    file_format=tfrecord,
    download_size=786.67 MiB,
    dataset_size=1.04 GiB,
    features=FeaturesDict({
        'image': Image(shape=(None, None, 3), dtype=uint8),
        'image/filename': Text(shape=(), dtype=string),
        'label': ClassLabel(shape=(), dtype=int64, num_classes=2),
    }),
    supervised_keys=('image', 'label'),
    disable_shuffling=False,
    nondeterministic_order=False,
    splits={
        'train': <SplitInfo num_examples=23262, num_shards=16>,
    },
    citation="""@Inproceedings (Conference){asirra-a-captcha-that-exploits-interest-aligned-manual-image-categorization,
author = {Elson, Jeremy and Douceur, John (JD) and Howell, Jon and Saul, Jared},
title = {Asirra: A CAPTCHA that Exploits Interest-Aligned Manual Image Categorization},
booktitle = {Proceedings of 14th ACM Conference on Computer and Communications Security (CCS)},
year = {2007},
month = {October},
publisher = {Association for Computing Machinery, Inc.},
url = {https://www.microsoft.com/en-us/research/publication/asirra-a-captcha-that-exploits-interest-aligned-manual-image-categorization/},
edition = {Proceedings of 14th ACM Conference on Computer and Communications Security (CCS)},
}""",
)
```

En esta fase se realizó una visualización preliminar de los datos de entrenamiento con el fin de verificar su correcta carga y etiquetado. Para ello, se emplearon dos métodos proporcionados por TensorFlow Datasets. El primero consistió en convertir una muestra de cinco elementos del conjunto de entrenamiento en un DataFrame, lo que permitió observar simultáneamente las imágenes y sus etiquetas correspondientes. El segundo método utilizó una función de visualización automática que presenta múltiples ejemplos junto con su clase asociada.



## Preprocesamiento de imágenes

El preprocesamiento es una etapa clave para garantizar un entrenamiento adecuado del modelo.

### Redimensionamiento

Todas las imágenes son redimensionadas a un tamaño fijo de  $100 \times 100$  píxeles. Este tamaño representa un equilibrio entre conservar información visual relevante y reducir el costo computacional del entrenamiento.

### Conversión a escala de grises

Las imágenes son convertidas a escala de grises, reduciendo el número de canales de color de tres (RGB) a uno. Esta decisión permite disminuir la complejidad del modelo y acelerar el entrenamiento, manteniendo suficiente información visual para distinguir entre las clases.

```
#Manipular y visualizar el set
#Lo pasamos a TAMANO_IMG (100x100) y a blanco y negro (solo para visualizar)
import matplotlib.pyplot as plt
import cv2

plt.figure(figsize=(20,20))

TAMANO_IMG=100

for i, (imagen, etiqueta) in enumerate(datos['train'].take(25)):
    imagen = cv2.resize(imagen.numpy(), (TAMANO_IMG, TAMANO_IMG))
    imagen = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
    plt.subplot(5, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(imagen, cmap='gray')
```



## Ajuste de forma

Cada imagen se reorganiza para tener la forma (100, 100, 1), donde el último valor indica un único canal de color. Esta estructura es necesaria para que TensorFlow procese correctamente las imágenes dentro de la red neuronal. Posteriormente, cada imagen procesada se almacenó junto con su etiqueta asociada en una lista destinada a los datos de entrenamiento.

```
#Variable que contendra todos los pares de los datos (imagen y etiqueta) ya modificados (blanco y negro, 100x100)
datos_entrenamiento = []

for i, (imagen, etiqueta) in enumerate(datos['train']): #Todos los datos
    imagen = cv2.resize(imagen.numpy(), (TAMANO_IMG, TAMANO_IMG))
    imagen = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
    imagen = imagen.reshape(TAMANO_IMG, TAMANO_IMG, 1) #Cambiar tamaño a 100,100,1
    datos_entrenamiento.append([imagen, etiqueta])
```

## Separación de variables

Los datos se separan en dos conjuntos:

- X: contiene las imágenes procesadas en forma de matrices de píxeles.

- Y: contiene las etiquetas correspondientes (0 para gato y 1 para perro).

```
#Preparar mis variables X (entradas) y y (etiquetas) separadas

X = [] #imagenes de entrada (píxeles)
y = [] #etiquetas (perro o gato)

for imagen, etiqueta in datos_entrenamiento:
    X.append(imagen)
    y.append(etiqueta)
```

## Normalización

Los valores de los píxeles, originalmente comprendidos entre 0 y 255, se normalizan a un rango de 0 a 1 dividiendo cada valor entre 255. Este paso mejora la estabilidad numérica del entrenamiento y facilita la convergencia del modelo.

```
#Normalizar los datos de las X (imagenes). Se pasan a numero flotante y dividen entre 255 para quedar de 0-1 en lugar de 0-255
import numpy as np

X = np.array(X).astype(float) / 255
```

## Definición de modelos de entrenamiento

Se diseñan y entrenan tres modelos distintos, con el objetivo de comparar su desempeño y analizar sus ventajas y desventajas.

### Modelo 1: Red neuronal densa

Este modelo utiliza únicamente capas densas:

- Recibe como entrada imágenes de  $100 \times 100$  píxeles en escala de grises.
- Utiliza una capa de aplanamiento (Flatten) para convertir la imagen en un vector.
- Dos capas densas intermedias con aproximadamente 150 neuronas cada una.



- Una capa de salida con una sola neurona y función de activación sigmoide, adecuada para clasificación binaria.

Este modelo sirve como referencia inicial, aunque no está optimizado para procesamiento de imágenes.

### **Modelo 2: Red neuronal convolucional (CNN)**

Este modelo incorpora capas convolucionales y de agrupamiento máximo (max pooling):

- Emplea tres bloques de convolución y *max pooling* con 32, 64 y 128 filtros respectivamente.
- Varias capas convolucionales con un número creciente de filtros.
- Capas de reducción de dimensionalidad mediante pooling.
- Una capa densa final con aproximadamente 100 neuronas.
- Capa de salida con activación sigmoide.

Este tipo de red es más adecuada para el reconocimiento de patrones visuales.

### **Modelo 3: CNN con Dropout**

Este modelo es similar al anterior, pero incluye una capa Dropout con una tasa de 0.5 antes de la capa densa final y utiliza una capa densa de 250 neuronas antes de la capa de salida. El objetivo de esta técnica es reducir el sobreajuste al desactivar aleatoriamente neuronas durante el entrenamiento.

```
| #Crear los modelos iniciales
| #Usan sigmoid como salida (en lugar de softmax) para mostrar como podria funcionar con dicha funcion de activacion.
| #Sigmoid regresa siempre datos entre 0 y 1. Realizamos el entrenamiento para al final considerar que si la respuesta se
| #acerca a 0, es un gato, y si se acerca a 1, es un perro.

modeloDenso = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(100, 100, 1)),
    tf.keras.layers.Dense(150, activation='relu'),
    tf.keras.layers.Dense(150, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

modeloCNN = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(100, 100, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

modeloCNN2 = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(100, 100, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(250, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

## Entrenamiento y evaluación inicial (sin aumento de datos)

Los modelos se entrenan utilizando:

- Optimizador Adam: debido a su buen desempeño general en tareas de clasificación y su capacidad de ajustar dinámicamente la tasa de aprendizaje.
- Función de pérdida binary crossentropy
- Métrica accuracy: para medir el porcentaje de predicciones correctas durante el entrenamiento y la validación.

Se separa un porcentaje de los datos para validación y se monitorea el entrenamiento mediante TensorBoard, lo que permite analizar gráficas de precisión y pérdida.

```
#Compilar modelos. Usar crossentropy binario ya que tenemos solo 2 opciones (perro o gato)
modeloDenso.compile(optimizer='adam',
                    loss='binary_crossentropy',
                    metrics=['accuracy'])

modeloCNN.compile(optimizer='adam',
                 loss='binary_crossentropy',
                 metrics=['accuracy'])

modeloCNN2.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
```

## Entrenamiento del modelo denso y registro con TensorBoard

En esta fase se realizó el entrenamiento del modelo de red neuronal densa utilizando los datos previamente preparados. Para el seguimiento del proceso de aprendizaje se empleó la herramienta TensorBoard, la cual permite registrar y visualizar métricas de entrenamiento a lo largo de las épocas.

### Configuración del entrenamiento

- Se utilizó un tamaño de lote (batch size) de 32 imágenes.
- Se definió un total de 100 épocas de entrenamiento.
- Se reservó automáticamente un 15 % de los datos para validación.
- Se incorporó TensorBoard mediante el arreglo de callbacks.

### Uso de TensorBoard

- Los resultados de cada época se almacenaron en un directorio específico.
- Se registraron métricas como precisión (accuracy) y función de pérdida (loss).
- Esta información permite generar gráficas para analizar el comportamiento del modelo.

### Análisis del desempeño del modelo denso

## Precisión (Accuracy)

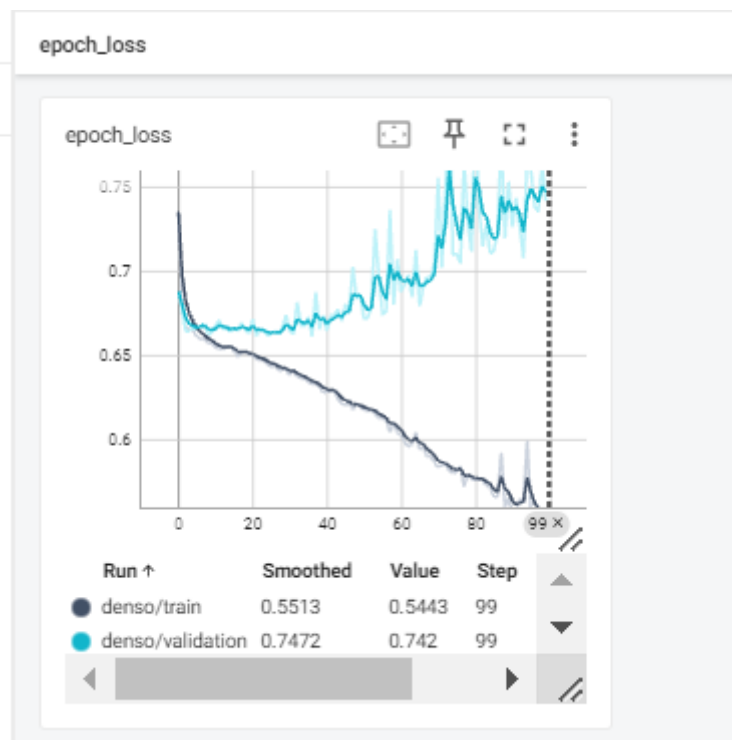
- La precisión del conjunto de entrenamiento aumenta de forma constante a lo largo de las épocas, alcanzando valores cercanos al 72–73 %.
- La precisión del conjunto de validación se mantiene significativamente más baja, alrededor del 58–60 %, sin mostrar una mejora sostenida.
- La divergencia creciente entre ambas curvas indica que el modelo aprende los datos de entrenamiento, pero no generaliza adecuadamente.



## Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye de manera progresiva, lo cual sugiere que el modelo optimiza correctamente sobre los datos que ve durante el aprendizaje.

- En contraste, la pérdida del conjunto de validación aumenta conforme avanzan las épocas.
- Este comportamiento evidencia que el modelo empeora su desempeño sobre datos no vistos.



## Entrenamiento de la CNN

Durante el entrenamiento del modelo CNN se observa un aumento progresivo de la precisión y una disminución de la pérdida en el conjunto de entrenamiento. En las primeras épocas, el conjunto de validación muestra un comportamiento similar, indicando una correcta capacidad de aprendizaje inicial.

```
tensorboardCNN = TensorBoard(log_dir='logs/cnn')
modeloCNN.fit(X, y, batch_size=32,
              validation_split=0.15,
              epochs=100,
              callbacks=[tensorboardCNN])
```

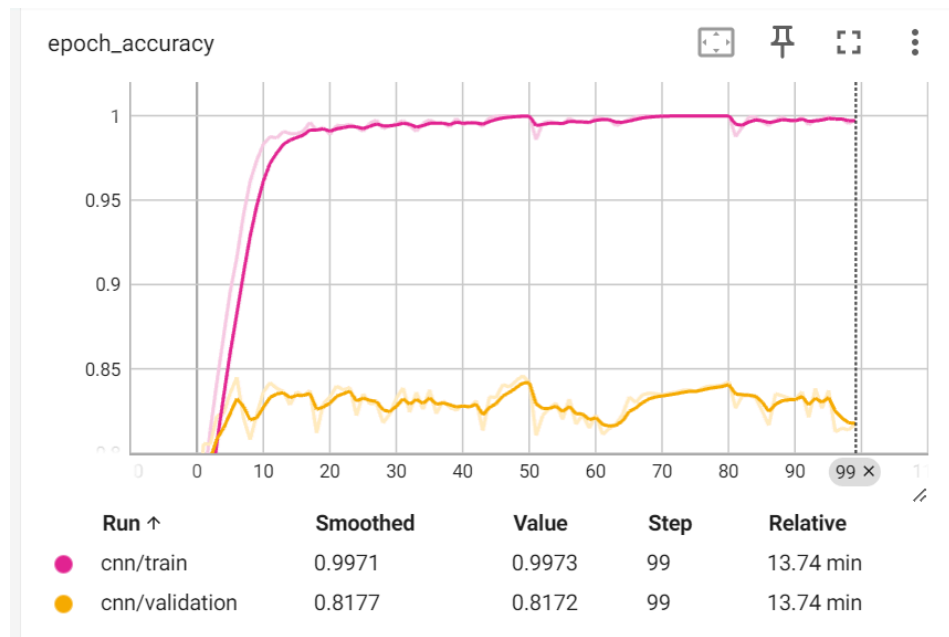
  

```
... Epoch 1/100
618/618 ————— 17s 19ms/step - accuracy: 0.5963 - loss: 0.6509 - val_accuracy: 0.7576 - val_loss: 0.5027
Epoch 2/100
618/618 ————— 8s 13ms/step - accuracy: 0.7725 - loss: 0.4800 - val_accuracy: 0.8057 - val_loss: 0.4343
Epoch 3/100
618/618 ————— 8s 13ms/step - accuracy: 0.8104 - loss: 0.4122 - val_accuracy: 0.8052 - val_loss: 0.4050
Epoch 4/100
618/618 ————— 8s 13ms/step - accuracy: 0.8382 - loss: 0.3582 - val_accuracy: 0.8221 - val_loss: 0.3854
Epoch 5/100
618/618 ————— 8s 13ms/step - accuracy: 0.8709 - loss: 0.3015 - val_accuracy: 0.8235 - val_loss: 0.3873
Epoch 6/100
618/618 ————— 8s 13ms/step - accuracy: 0.9000 - loss: 0.2413 - val_accuracy: 0.8355 - val_loss: 0.3953
Epoch 7/100
618/618 ————— 10s 13ms/step - accuracy: 0.9168 - loss: 0.1973 - val_accuracy: 0.8450 - val_loss: 0.4037
Epoch 8/100
618/618 ————— 8s 13ms/step - accuracy: 0.9427 - loss: 0.1436 - val_accuracy: 0.8209 - val_loss: 0.5270
Epoch 9/100
618/618 ————— 8s 13ms/step - accuracy: 0.9599 - loss: 0.1031 - val_accuracy: 0.8080 - val_loss: 0.6270
Epoch 10/100
618/618 ————— 8s 13ms/step - accuracy: 0.9721 - loss: 0.0745 - val_accuracy: 0.8229 - val_loss: 0.6174
Epoch 11/100
618/618 ————— 8s 14ms/step - accuracy: 0.9890 - loss: 0.0360 - val_accuracy: 0.8364 - val_loss: 0.6288
Epoch 12/100
618/618 ————— 8s 13ms/step - accuracy: 0.9886 - loss: 0.0368 - val_accuracy: 0.8418 - val_loss: 0.7123
```

## Análisis del desempeño del modelo convolucional

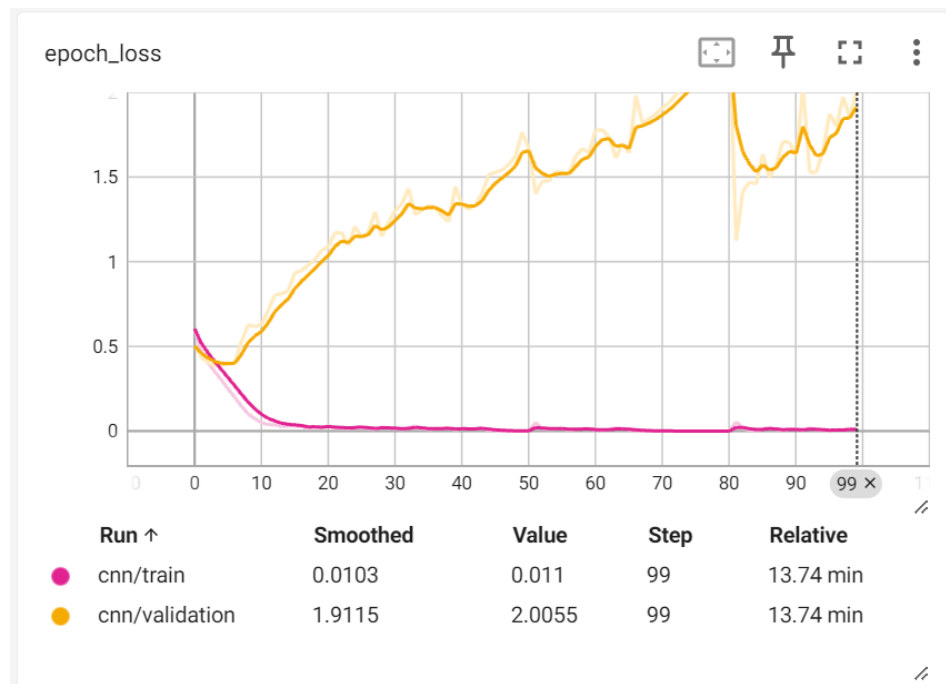
### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye rápidamente hasta valores cercanos a cero.
- En contraste, la pérdida del conjunto de validación aumenta de forma progresiva, alcanzando valores elevados al final del entrenamiento.
- Esta divergencia indica que el modelo aprende muy bien los datos de entrenamiento, pero presenta un deterioro significativo al evaluar datos no vistos.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento alcanza valores cercanos al 99 %, lo que evidencia una alta capacidad de ajuste sobre los datos de entrenamiento.
- La precisión del conjunto de validación se mantiene alrededor del 81–83 %, mostrando un desempeño considerablemente inferior.
- La brecha constante entre ambas curvas confirma la falta de generalización del modelo.



### Interpretación del comportamiento

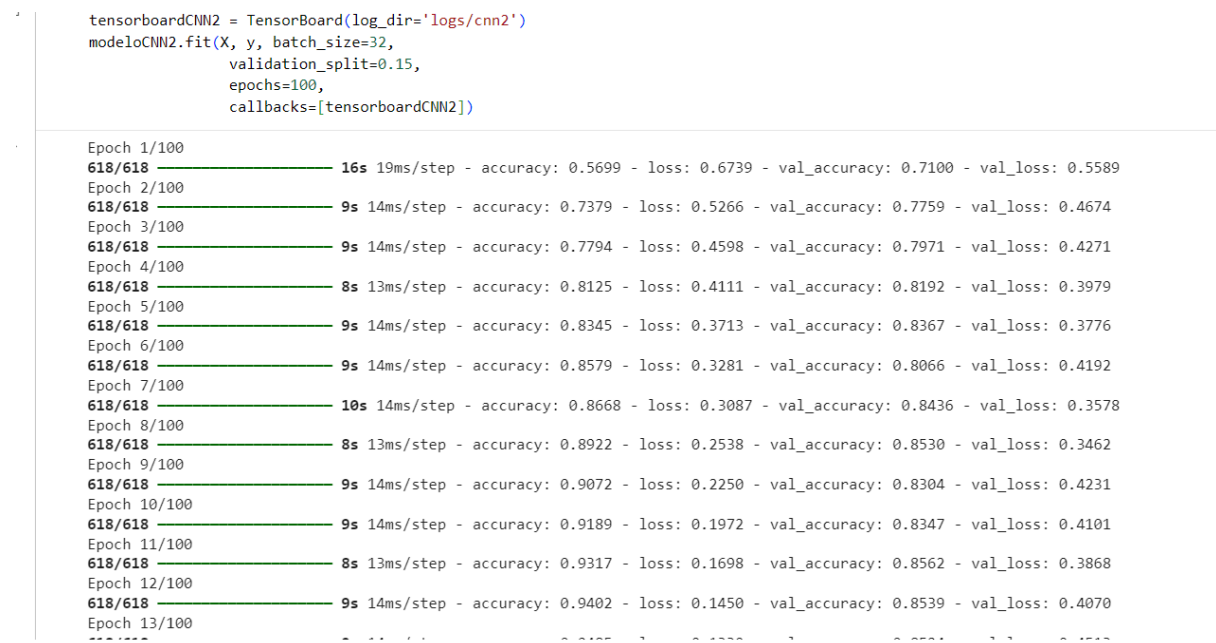
- El modelo convolucional extrae características relevantes de las imágenes con gran eficacia.
- Sin embargo, el rápido descenso de la pérdida de entrenamiento y el aumento simultáneo de la pérdida de validación son indicativos de sobreajuste severo.
- El modelo memoriza patrones específicos del conjunto de entrenamiento en lugar de aprender representaciones generalizables.

Aunque el modelo convolucional logra una alta precisión en el conjunto de entrenamiento y mejora notablemente el desempeño respecto al modelo denso, los resultados evidencian un sobreajuste pronunciado. Esto resalta la necesidad de incorporar técnicas adicionales como aumento de datos, regularización más agresiva o una reducción en la complejidad del modelo para mejorar su capacidad de generalización.



## Entrenamiento del modelo CNN2

El modelo CNN2 muestra un aprendizaje progresivo tanto en el conjunto de entrenamiento como en el de validación durante las primeras épocas. La precisión de validación alcanza valores cercanos al 85% mientras la pérdida disminuye, lo que indica una mejor capacidad de generalización inicial en comparación con el modelo anterior.

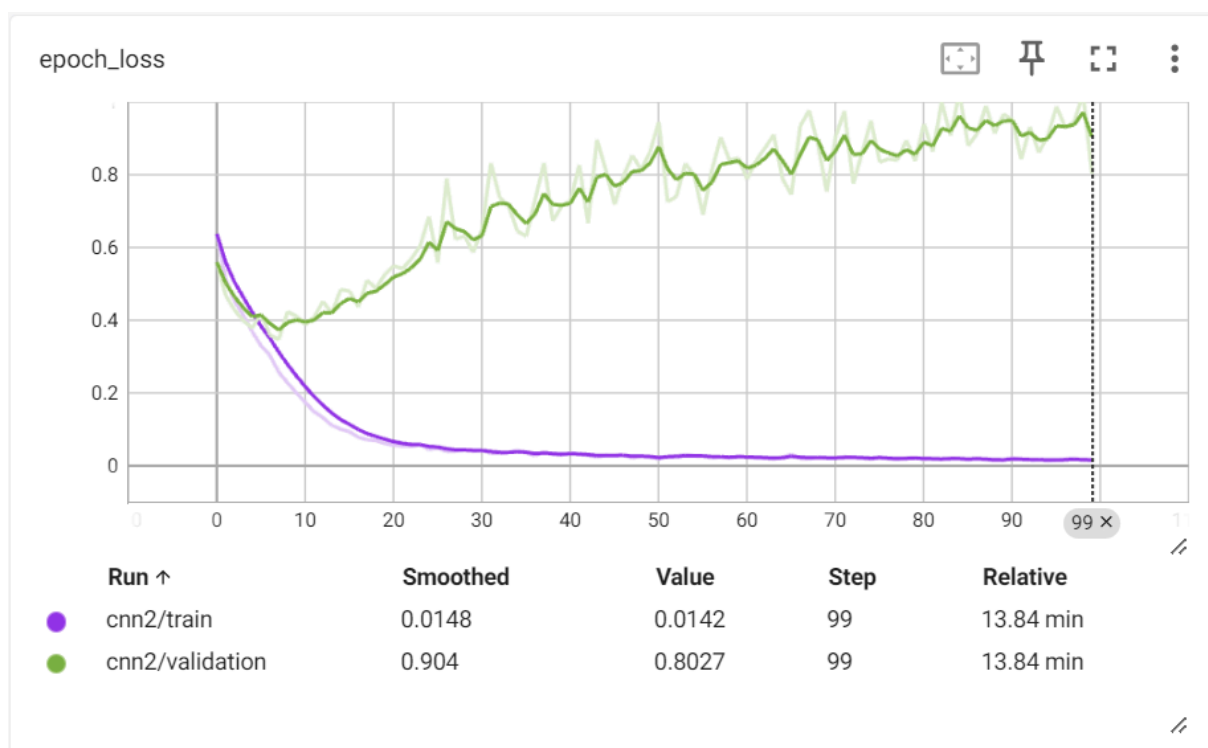


## Análisis del desempeño del modelo convolucional con regularización

### Función de pérdida (Loss)

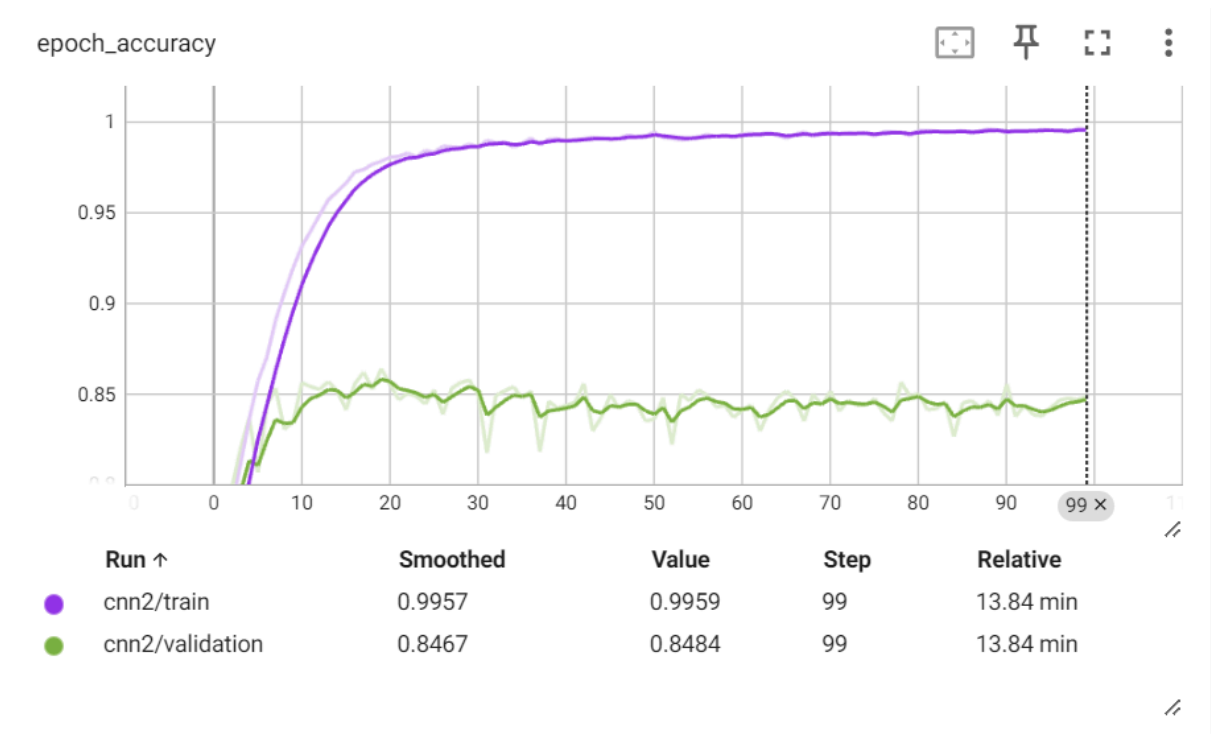
- La pérdida del conjunto de entrenamiento disminuye de manera sostenida hasta valores cercanos a cero.

- La pérdida del conjunto de validación muestra una tendencia creciente, aunque con valores máximos inferiores a los observados en el modelo convolucional sin regularización.
- Este comportamiento indica que el dropout atenúa parcialmente el sobreajuste, aunque no logra eliminarlo por completo.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento alcanza valores cercanos al 99.5 %, lo que refleja una alta capacidad de ajuste del modelo.
- La precisión del conjunto de validación se estabiliza alrededor del 84–85 %, superando el desempeño del modelo convolucional sin dropout.
- La brecha entre entrenamiento y validación es menor, lo que sugiere una mejora en la generalización.



### Interpretación del comportamiento

- La incorporación de dropout reduce la dependencia excesiva de neuronas individuales durante el entrenamiento.
- El modelo mantiene una alta capacidad de aprendizaje sin sacrificar significativamente el desempeño en datos no vistos.
- A pesar de la mejora, persisten indicios de sobreajuste, evidenciados por la diferencia entre las curvas de pérdida.

El modelo convolucional con dropout presenta un mejor equilibrio entre aprendizaje y generalización en comparación con las arquitecturas previas. Si bien no elimina completamente el sobreajuste, constituye una mejora significativa y demuestra la efectividad de la regularización. Estos resultados respaldan la incorporación de técnicas adicionales, como el aumento de datos, para optimizar aún más el desempeño del modelo.

## Aplicación de aumento de datos (Data Augmentation)

En esta etapa se aplicó la técnica de aumento de datos con el objetivo de incrementar la variabilidad del conjunto de entrenamiento sin necesidad de recolectar nuevas imágenes. Para ello, se utilizó la clase ImageDataGenerator de Keras, la cual permite generar transformaciones aleatorias sobre las imágenes originales durante el entrenamiento.

### Transformaciones aplicadas

Se configuraron las siguientes operaciones de aumento de datos:

- Rotación aleatoria de hasta 30 grados.
- Desplazamientos horizontales y verticales de hasta el 20 %.
- Transformaciones de cizallamiento (shear).
- Escalado aleatorio (zoom in y zoom out).
- Inversión horizontal y vertical de las imágenes..

● Realizar el aumento de datos con varias transformaciones. Al final, graficar 10 como ejemplo

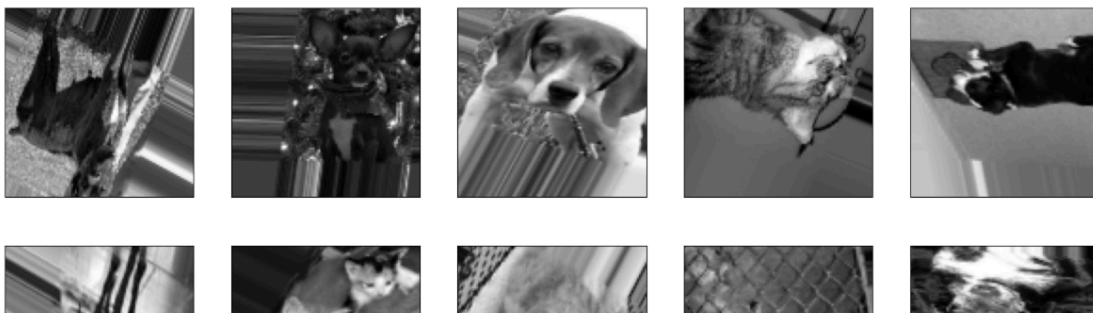
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=15,
    zoom_range=[0.7, 1.4],
    horizontal_flip=True,
    vertical_flip=True
)

datagen.fit(X)

plt.figure(figsize=(20,8))

for imagen, etiqueta in datagen.flow(X, y, batch_size=10, shuffle=False):
    for i in range(10):
        plt.subplot(2, 5, i+1)
        plt.xticks([])
        plt.yticks([])
        plt.imshow(imagen[i].reshape(100, 100), cmap="gray")
    break
```



Este proceso permite simular una mayor diversidad de datos sin necesidad de recolectar nuevas imágenes.

### **Definición de modelos con aumento de datos**

En esta etapa se definieron nuevamente tres arquitecturas de redes neuronales, las cuales serán entrenadas utilizando aumento de datos. Las estructuras de los modelos se mantienen iguales a las versiones previas, con el fin de evaluar exclusivamente el impacto del aumento de datos en el desempeño y la generalización.

#### **Modelo denso con aumento de datos**

- Recibe imágenes de entrada de  $100 \times 100$  píxeles en escala de grises.
- Utiliza una capa de aplanamiento (Flatten).
- Incluye dos capas densas de 150 neuronas con activación ReLU.
- La capa de salida contiene una neurona con activación sigmoide para clasificación binaria.

#### **Modelo convolucional con aumento de datos**

- Emplea tres capas convolucionales con 32, 64 y 128 filtros respectivamente.
- Cada capa convolucional es seguida por una capa de max pooling para la reducción espacial.
- Finaliza con una capa densa de 100 neuronas y una capa de salida sigmoide.

#### **Modelo convolucional con dropout y aumento de datos**

- Presenta una arquitectura similar al modelo convolucional estándar.

- Incorpora una capa de dropout con una tasa de 0.5 antes de las capas densas.
- Utiliza una capa densa de 250 neuronas para aumentar la capacidad de representación.
- La capa de salida emplea activación sigmoide.

```
modeloDense_AD = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(100, 100, 1)),
    tf.keras.layers.Dense(150, activation='relu'),
    tf.keras.layers.Dense(150, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

modeloCNN_AD = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(100, 100, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

modeloCNN2_AD = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(100, 100, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(250, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

## Compilación de los modelos

En esta fase se compilaron los modelos definidos para el entrenamiento con aumento de datos. Para todos los casos se utilizó el optimizador Adam, junto con la función de pérdida binary crossentropy, adecuada para problemas de clasificación binaria. La métrica seleccionada para evaluar el desempeño fue la precisión (accuracy).

```
modeloDenso_AD.compile(optimizer='adam',  
                        loss='binary_crossentropy',  
                        metrics=['accuracy'])  
  
modeloCNN_AD.compile(optimizer='adam',  
                     loss='binary_crossentropy',  
                     metrics=['accuracy'])  
  
modeloCNN2_AD.compile(optimizer='adam',  
                      loss='binary_crossentropy',  
                      metrics=['accuracy'])
```

## Separación de datos de entrenamiento y validación

Con el fin de evaluar correctamente el desempeño del modelo, el conjunto de datos se dividió manualmente en dos subconjuntos:

- **Datos de entrenamiento:** aproximadamente el 85 % del total de imágenes.
- **Datos de validación:** el 15 % restante.

Esta separación se realizó tanto para las imágenes de entrada como para sus etiquetas correspondientes, garantizando la coherencia entre ambos conjuntos.

```
#Separar los datos de entrenamiento y los datos de pruebas en variables diferentes  
  
len(X) * .85 #19700  
len(X) - 19700 #3562  
  
X_entrenamiento = X[:19700]  
X_validacion = X[19700:]  
  
y_entrenamiento = y[:19700]  
y_validacion = y[19700:]
```

## Generación de datos aumentados

Para el entrenamiento se utilizó el generador de aumento de datos previamente definido. Mediante la función flow, se creó un iterador que produce lotes de imágenes transformadas junto con sus etiquetas. Este iterador fue empleado como entrada del proceso de

entrenamiento del modelo, permitiendo aplicar las transformaciones de manera dinámica en cada época.

```
#Usar la funcion flow del generador para crear un iterador que podamos enviar como entrenamiento a la funcion FIT del modelo
data_gen_entrenamiento = datagen.flow(X_entrenamiento, y_entrenamiento, batch_size=32)
```

## Entrenamiento del modelo Denso con aumento de datos (Dense\_AD)

Durante el entrenamiento del modelo Dense\_AD se observa que tanto la precisión como la función de pérdida permanecen prácticamente constantes a lo largo de las épocas. La precisión de entrenamiento se sitúa alrededor del 50% y la precisión de validación en torno al 49%, mientras que la pérdida se mantiene cercana a 0.69 en ambos conjuntos.

```
tensorboardDense_AD = TensorBoard(log_dir='logs/dense_AD')

modeloDense_AD.fit(
    data_gen_entrenamiento,
    epochs=100, batch_size=32,
    validation_data=(X_validacion, y_validacion),
    steps_per_epoch=int(np.ceil(len(X_entrenamiento) / float(32))),
    validation_steps=int(np.ceil(len(X_validacion) / float(32))),
    callbacks=[tensorboardDense_AD]
)
```

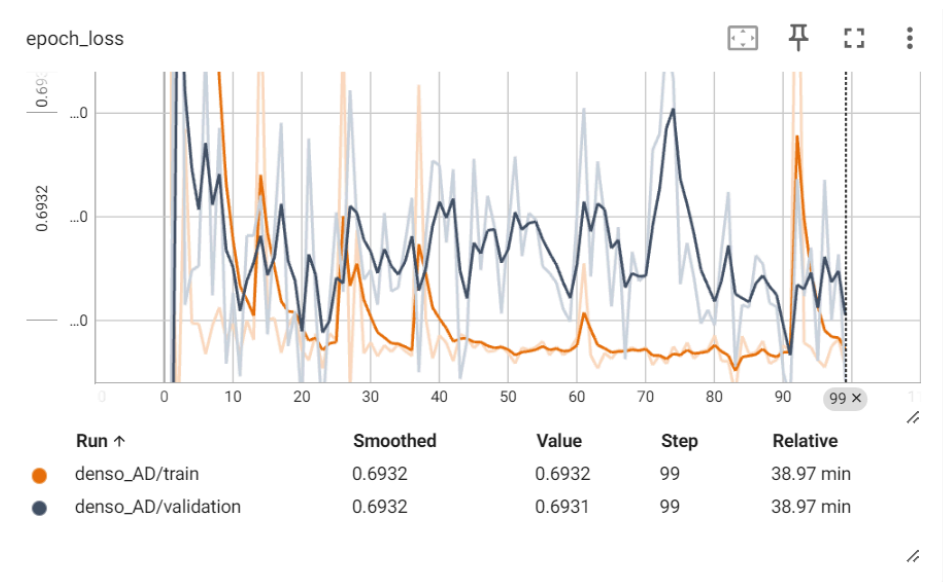
```
*** Epoch 1/100
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your 'PyDataset' class
self.warn_if_super_not_called()
616/616 23s 35ms/step - accuracy: 0.5025 - loss: 0.7660 - val_accuracy: 0.4916 - val_loss: 0.6934
Epoch 2/100
616/616 20s 33ms/step - accuracy: 0.5032 - loss: 0.6934 - val_accuracy: 0.4919 - val_loss: 0.6932
Epoch 3/100
616/616 21s 34ms/step - accuracy: 0.5038 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6935
Epoch 4/100
616/616 20s 32ms/step - accuracy: 0.4949 - loss: 0.6931 - val_accuracy: 0.4921 - val_loss: 0.6934
Epoch 5/100
616/616 22s 34ms/step - accuracy: 0.5052 - loss: 0.6931 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 6/100
616/616 21s 34ms/step - accuracy: 0.4924 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6933
Epoch 7/100
616/616 20s 32ms/step - accuracy: 0.4980 - loss: 0.6933 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 8/100
616/616 21s 34ms/step - accuracy: 0.5042 - loss: 0.6933 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 9/100
616/616 21s 34ms/step - accuracy: 0.5021 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6933
Epoch 10/100
616/616 20s 32ms/step - accuracy: 0.4972 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6933
Epoch 11/100
616/616 21s 34ms/step - accuracy: 0.5029 - loss: 0.6950 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 12/100
616/616 21s 34ms/step - accuracy: 0.5016 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 13/100
616/616 20s 32ms/step - accuracy: 0.4996 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6933
Epoch 14/100
616/616 21s 34ms/step - accuracy: 0.5039 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6933
Epoch 15/100
616/616 21s 34ms/step - accuracy: 0.5061 - loss: 0.6946 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 16/100
616/616 20s 33ms/step - accuracy: 0.5071 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 17/100
616/616 21s 35ms/step - accuracy: 0.5051 - loss: 0.6931 - val_accuracy: 0.4921 - val_loss: 0.6932
Epoch 18/100
616/616 22s 35ms/step - accuracy: 0.4889 - loss: 0.6932 - val_accuracy: 0.4921 - val_loss: 0.6933
```

## Análisis del desempeño del modelo denso con aumento de datos



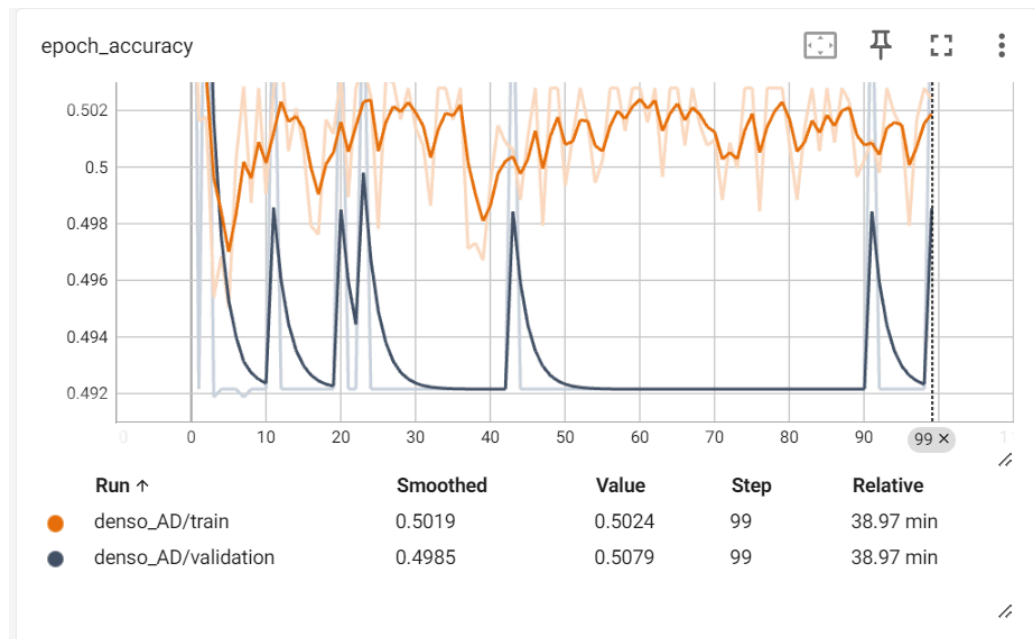
## Función de pérdida (Loss)

- Tanto la pérdida de entrenamiento como la de validación se mantienen alrededor de 0.693 durante todas las épocas.
- Este valor corresponde al comportamiento esperado de un clasificador binario que realiza predicciones cercanas al azar.
- No se observa una tendencia descendente sostenida en ninguna de las curvas.}



## Precisión (Accuracy)

- La precisión del conjunto de entrenamiento oscila alrededor del 50 %.
- La precisión del conjunto de validación se mantiene en valores similares, sin mejoras significativas.
- Ambas curvas permanecen prácticamente superpuestas, lo que indica ausencia de aprendizaje efectivo.



### Interpretación del comportamiento

- El modelo no logra extraer información discriminativa de las imágenes.
- El aumento de datos introduce variabilidad adicional que el modelo denso no es capaz de aprovechar.
- La arquitectura carece de mecanismos para capturar relaciones espaciales presentes en imágenes.

El modelo denso con aumento de datos no presenta mejoras respecto a su versión sin aumento. El desempeño cercano al azar confirma que este tipo de arquitectura no es adecuada para tareas de clasificación de imágenes, incluso cuando se aplican técnicas de aumento de datos. Estos resultados refuerzan la necesidad de emplear redes neuronales convolucionales para este problema.

### Entrenamiento del modelo CNN con aumento de datos (CNN\_AD)

El modelo CNN\_AD presenta una mejora progresiva tanto en la precisión como en la función de pérdida a lo largo de las primeras épocas, evidenciando un aprendizaje estable. La precisión de validación alcanza valores cercanos al 78–80%, mientras que la pérdida de validación disminuye de forma consistente.

A diferencia de los modelos previos sin aumento de datos, no se observa una divergencia temprana marcada entre entrenamiento y validación, lo que indica una mejor capacidad de generalización. Aunque la precisión de entrenamiento continúa aumentando, el comportamiento del conjunto de validación se mantiene relativamente estable, reduciendo el efecto del sobreajuste.

```

tensorboardCNN_AD = TensorBoard(log_dir='logs-new/cnn_AD')

modeloCNN_AD.fit(
    data_gen_entrenamiento,
    epochs=150, batch_size=32,
    validation_data=(X_validacion, y_validacion),
    steps_per_epoch=int(np.ceil(len(X_entrenamiento) / float(32))),
    validation_steps=int(np.ceil(len(X_validacion) / float(32))),
    callbacks=[tensorboardCNN_AD]
)

```

```

*** Epoch 1/150
616/616 ----- 32s 47ms/step - accuracy: 0.5247 - loss: 0.6898 - val_accuracy: 0.5811 - val_loss: 0.6799
Epoch 2/150
616/616 ----- 25s 41ms/step - accuracy: 0.5939 - loss: 0.6702 - val_accuracy: 0.6533 - val_loss: 0.6321
Epoch 3/150
616/616 ----- 26s 42ms/step - accuracy: 0.6148 - loss: 0.6538 - val_accuracy: 0.6755 - val_loss: 0.6034
Epoch 4/150
616/616 ----- 41s 41ms/step - accuracy: 0.6433 - loss: 0.6324 - val_accuracy: 0.6157 - val_loss: 0.6414
Epoch 5/150
616/616 ----- 25s 41ms/step - accuracy: 0.6589 - loss: 0.6171 - val_accuracy: 0.7134 - val_loss: 0.5515
Epoch 6/150
616/616 ----- 25s 40ms/step - accuracy: 0.6644 - loss: 0.6105 - val_accuracy: 0.7117 - val_loss: 0.5634
Epoch 7/150
616/616 ----- 25s 41ms/step - accuracy: 0.6815 - loss: 0.5956 - val_accuracy: 0.7330 - val_loss: 0.5262
Epoch 8/150
616/616 ----- 26s 41ms/step - accuracy: 0.6990 - loss: 0.5805 - val_accuracy: 0.6749 - val_loss: 0.5841
Epoch 9/150
616/616 ----- 26s 42ms/step - accuracy: 0.7038 - loss: 0.5718 - val_accuracy: 0.6940 - val_loss: 0.5956
Epoch 10/150
616/616 ----- 26s 43ms/step - accuracy: 0.7036 - loss: 0.5673 - val_accuracy: 0.7414 - val_loss: 0.5132
Epoch 11/150
616/616 ----- 26s 42ms/step - accuracy: 0.7098 - loss: 0.5611 - val_accuracy: 0.6923 - val_loss: 0.6088
Epoch 12/150
616/616 ----- 26s 42ms/step - accuracy: 0.7175 - loss: 0.5508 - val_accuracy: 0.7558 - val_loss: 0.4937
Epoch 13/150
616/616 ----- 26s 41ms/step - accuracy: 0.7287 - loss: 0.5426 - val_accuracy: 0.7636 - val_loss: 0.4909
Epoch 14/150
616/616 ----- 25s 41ms/step - accuracy: 0.7325 - loss: 0.5359 - val_accuracy: 0.7647 - val_loss: 0.4782
Epoch 15/150
616/616 ----- 25s 41ms/step - accuracy: 0.7373 - loss: 0.5296 - val_accuracy: 0.7695 - val_loss: 0.4707
Epoch 16/150
616/616 ----- 26s 41ms/step - accuracy: 0.7398 - loss: 0.5212 - val_accuracy: 0.7779 - val_loss: 0.4701
Epoch 17/150
616/616 ----- 25s 41ms/step - accuracy: 0.7465 - loss: 0.5194 - val_accuracy: 0.7524 - val_loss: 0.5209
Epoch 18/150
616/616 ----- 26s 41ms/step - accuracy: 0.7496 - loss: 0.5098 - val_accuracy: 0.7942 - val_loss: 0.4466
Epoch 19/150
616/616 ----- 41s 42ms/step - accuracy: 0.7565 - loss: 0.5001 - val_accuracy: 0.7906 - val_loss: 0.4528
Epoch 20/150
616/616 ----- 26s 42ms/step - accuracy: 0.7608 - loss: 0.4922 - val_accuracy: 0.7743 - val_loss: 0.4725
Epoch 21/150
616/616 ----- 26s 42ms/step - accuracy: 0.7608 - loss: 0.4922 - val_accuracy: 0.7743 - val_loss: 0.4725

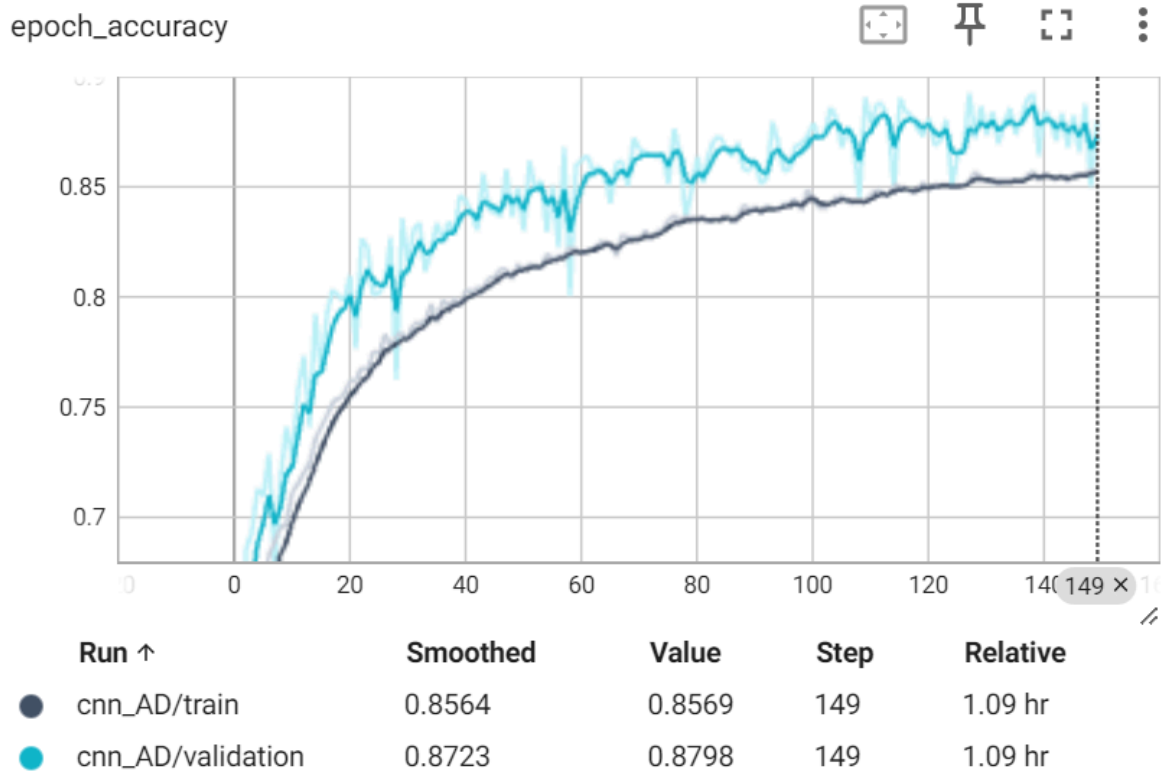
```

## Análisis del desempeño del modelo convolucional con aumento de datos

### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento aumenta de manera progresiva hasta alcanzar aproximadamente 85–86 %.
- La precisión del conjunto de validación se mantiene ligeramente por encima de la de entrenamiento, alcanzando valores cercanos al 88 %.

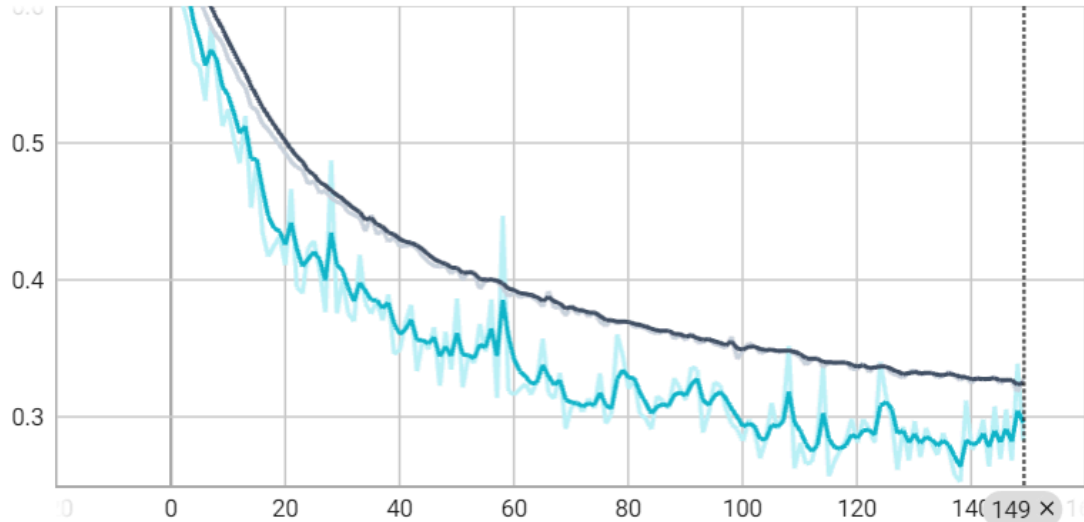
- La cercanía entre ambas curvas, sin divergencias significativas, indica una buena capacidad de generalización.



### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye de forma constante a lo largo de las épocas.
- La pérdida del conjunto de validación presenta una tendencia similar y se mantiene incluso por debajo de la pérdida de entrenamiento.
- Este comportamiento sugiere un proceso de aprendizaje estable y sin indicios de sobreajuste.

epoch\_loss



| Run ↑               | Smoothed | Value  | Step | Relative |
|---------------------|----------|--------|------|----------|
| ● cnn_AD/train      | 0.3249   | 0.3271 | 149  | 1.09 hr  |
| ● cnn_AD/validation | 0.2958   | 0.2825 | 149  | 1.09 hr  |

### Interpretación del comportamiento

- El aumento de datos aporta diversidad suficiente para evitar la memorización del conjunto de entrenamiento.
- El modelo logra aprender representaciones robustas de las imágenes sin necesidad de una regularización adicional agresiva.
- La arquitectura convolucional resulta adecuada para aprovechar las transformaciones aplicadas a los datos.

El modelo convolucional con aumento de datos presenta un desempeño sólido y equilibrado, con alta precisión y pérdidas decrecientes tanto en entrenamiento como en validación. Estos resultados confirman que el aumento de datos, combinado con redes convolucionales, es una estrategia efectiva para mejorar la generalización en problemas de clasificación de imágenes.

## Entrenamiento del modelo CNN2 con aumento de datos (CNN2\_AD)

El modelo CNN2\_AD muestra una mejora progresiva y estable en la precisión tanto de entrenamiento como de validación durante las primeras épocas. La precisión de validación alcanza valores cercanos al 79–80%, acompañada de una disminución consistente de la función de pérdida.

Gracias al uso de aumento de datos, no se observa un sobreajuste marcado, manteniéndose un comportamiento equilibrado entre entrenamiento y validación. No obstante, el rendimiento máximo alcanzado es ligeramente inferior al del modelo CNN2 sin aumento de datos.

```

tensorboardCNN2_AD = TensorBoard(log_dir='logs/cnn2_AD')

modeloCNN2_AD.fit(
    data_gen_entrenamiento,
    epochs=100, batch_size=32,
    validation_data=(X_validation, y_validation),
    steps_per_epoch=int(np.ceil(len(X_entrenamiento) / float(32))),
    validation_steps=int(np.ceil(len(X_validation) / float(32))),
    callbacks=[tensorboardCNN2_AD]
)

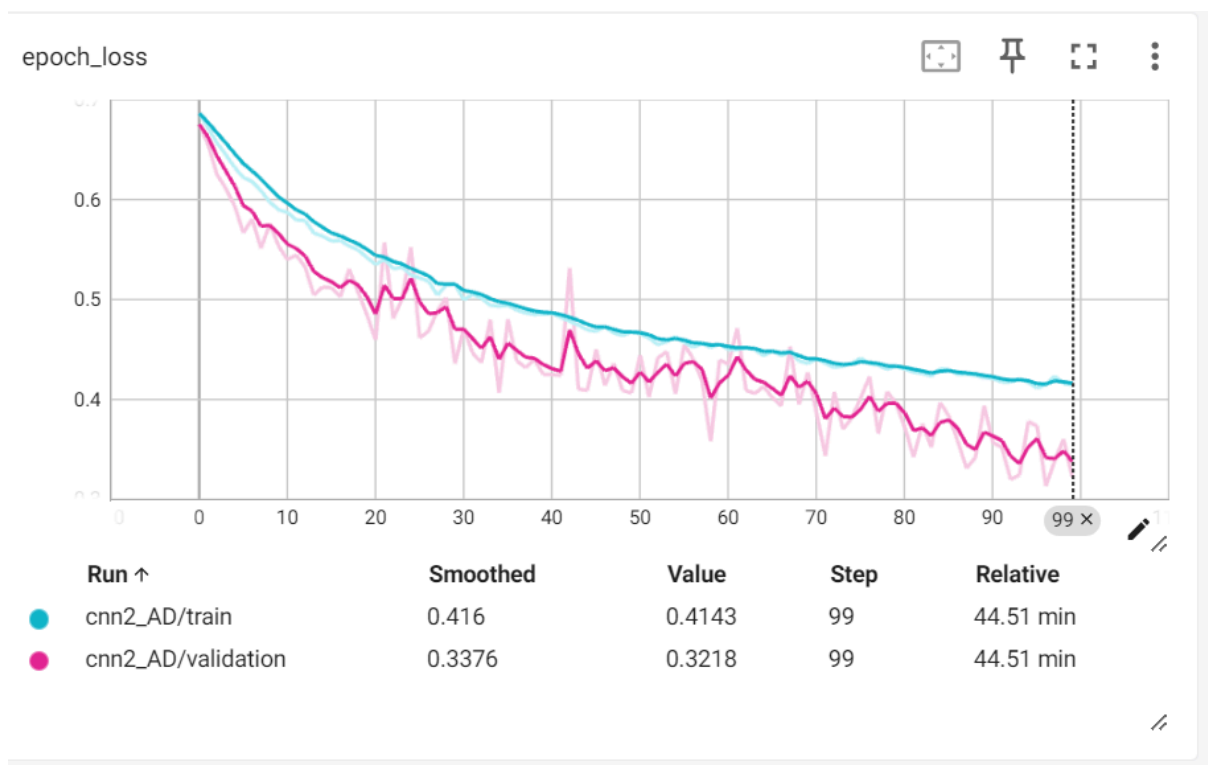
```

| Epoch        | Time          | Accuracy | Loss   | Val Accuracy | Val Loss |
|--------------|---------------|----------|--------|--------------|----------|
| Epoch 1/100  | 32s 46ms/step | 0.5286   | 0.6934 | 0.5752       | 0.6753   |
| Epoch 2/100  | 24s 40ms/step | 0.5792   | 0.6748 | 0.6224       | 0.6550   |
| Epoch 3/100  | 41s 39ms/step | 0.6049   | 0.6610 | 0.6541       | 0.6252   |
| Epoch 4/100  | 24s 40ms/step | 0.6281   | 0.6491 | 0.6700       | 0.6113   |
| Epoch 5/100  | 24s 40ms/step | 0.6434   | 0.6336 | 0.6755       | 0.5939   |
| Epoch 6/100  | 24s 40ms/step | 0.6447   | 0.6287 | 0.7035       | 0.5667   |
| Epoch 7/100  | 41s 40ms/step | 0.6608   | 0.6196 | 0.6881       | 0.5803   |
| Epoch 8/100  | 24s 40ms/step | 0.6629   | 0.6123 | 0.7198       | 0.5511   |
| Epoch 9/100  | 25s 40ms/step | 0.6813   | 0.5969 | 0.6988       | 0.5752   |
| Epoch 10/100 | 25s 40ms/step | 0.6894   | 0.5908 | 0.7150       | 0.5534   |
| Epoch 11/100 | 25s 41ms/step | 0.6976   | 0.5812 | 0.7280       | 0.5396   |
| Epoch 12/100 | 41s 41ms/step | 0.6973   | 0.5781 | 0.7212       | 0.5443   |
| Epoch 13/100 | 25s 41ms/step | 0.6969   | 0.5805 | 0.7330       | 0.5323   |
| Epoch 14/100 | 25s 40ms/step | 0.7111   | 0.5665 | 0.7524       | 0.5043   |
| Epoch 15/100 | 25s 40ms/step | 0.7081   | 0.5640 | 0.7493       | 0.5123   |
| Epoch 16/100 | 25s 40ms/step | 0.7075   | 0.5645 | 0.7473       | 0.5115   |
| Epoch 17/100 | 25s 40ms/step | 0.7151   | 0.5605 | 0.7617       | 0.5024   |
| Epoch 18/100 | 41s 40ms/step | 0.7165   | 0.5550 | 0.7350       | 0.5302   |
| Epoch 19/100 | 24s 39ms/step | 0.7195   | 0.5524 | 0.7487       | 0.5076   |
| Epoch 20/100 | 24s 39ms/step | 0.7326   | 0.5397 | 0.7743       | 0.4849   |
| Epoch 21/100 | 24s 40ms/step | 0.7323   | 0.5314 | 0.7920       | 0.4594   |

## Análisis del desempeño del modelo convolucional con aumento de datos

### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye de manera progresiva a lo largo de las épocas.
- La pérdida del conjunto de validación presenta una tendencia similar y se mantiene incluso por debajo de la pérdida de entrenamiento.
- La cercanía entre ambas curvas indica un proceso de aprendizaje estable y sin señales de sobreajuste.

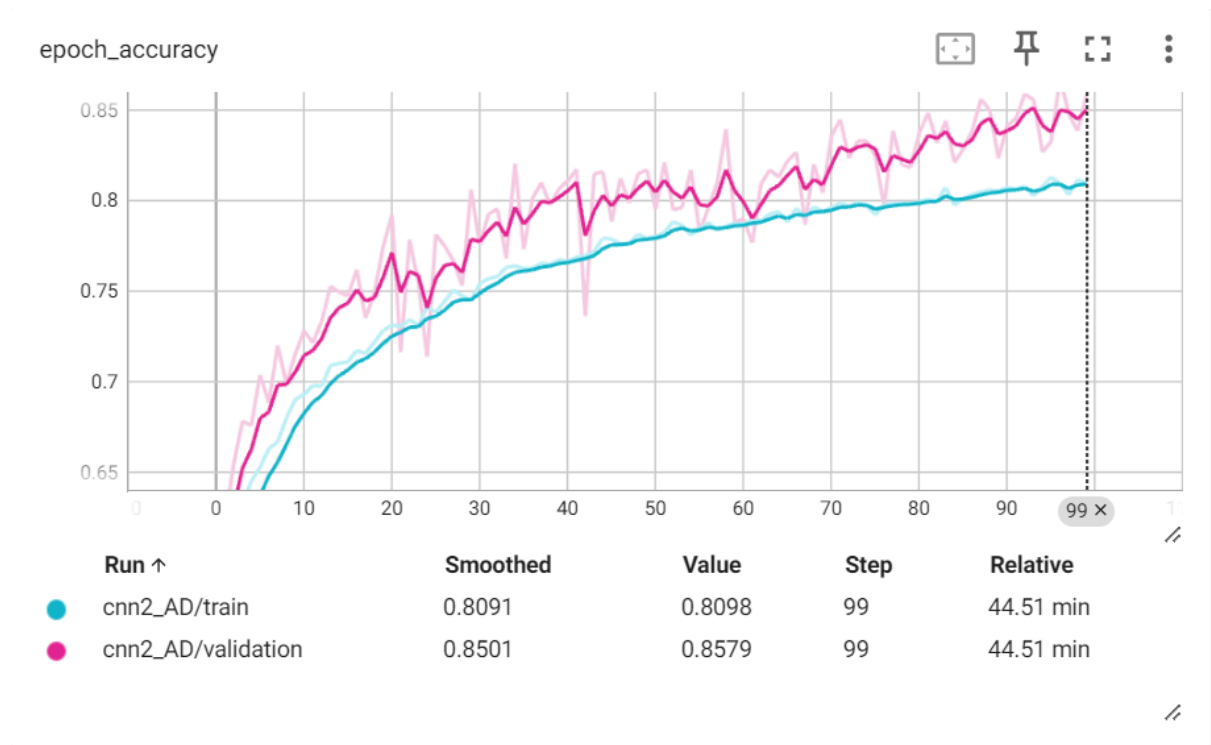


### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento alcanza valores cercanos al 81 %.



- La precisión del conjunto de validación supera ligeramente a la de entrenamiento, alcanzando aproximadamente 85–86 %.
- Este comportamiento sugiere una adecuada capacidad de generalización del modelo.



### Interpretación del comportamiento

- El aumento de datos introduce variabilidad suficiente para evitar que el modelo memorice los datos de entrenamiento.
- La combinación de aumento de datos y dropout contribuye a una mejor regularización del modelo.
- El modelo logra aprender representaciones robustas y generalizables de las imágenes.

El modelo convolucional con dropout y aumento de datos presenta el mejor desempeño entre todas las arquitecturas evaluadas. La ausencia de sobreajuste, junto con una alta precisión en

el conjunto de validación, confirma que la combinación de redes convolucionales, regularización y aumento de datos es efectiva para el problema de clasificación de perros y gatos.

### Resultados con aumento de datos

- El modelo denso no muestra mejoras significativas.
- La CNN con aumento de datos presenta un comportamiento más estable, con curvas de entrenamiento y validación similares.
- La CNN con Dropout y aumento ofrece resultados muy cercanos a la CNN normal, sin una mejora notable.

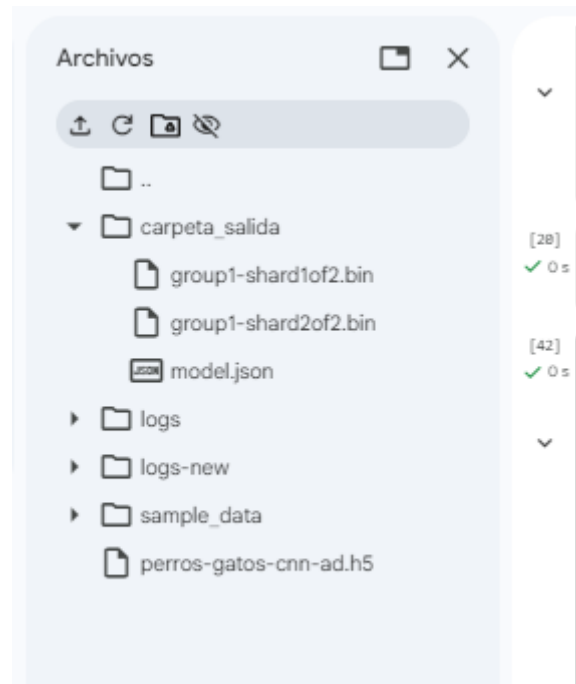
Como conclusión, el modelo que presenta mejor equilibrio entre precisión y generalización es la red neuronal convolucional con aumento de datos.

### Exportación del modelo

El modelo seleccionado se guarda y se convierte al formato TensorFlow.js, permitiendo su uso en aplicaciones web directamente desde el navegador.

```
Requirement already satisfied: etils[epath,epy] in /usr/local/lib/python3.12/dist-packages (from orbax-checkpoint->flax:
!mkdir carpeta_salida

!tensorflowjs_converter --input_format keras perros-gatos-cnn-ad.h5 carpeta_salida
```



## Implementación en una aplicación web

El modelo se integra en una aplicación web que:

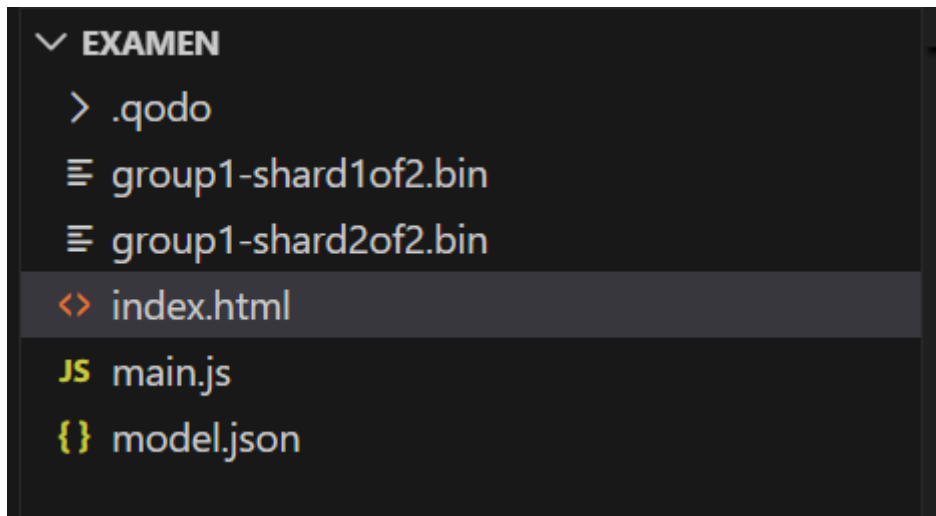
- Captura imágenes en tiempo real desde la cámara del dispositivo.
- Preprocesa cada imagen replicando exactamente los pasos del entrenamiento (redimensionamiento, normalización y forma)
- Realiza predicciones en tiempo real para clasificar si la imagen corresponde a un perro o un gato.

Para permitir el acceso a la cámara en dispositivos móviles, la aplicación se ejecuta en un entorno con HTTP, garantizando la compatibilidad con navegadores modernos.

## Integración del modelo de aprendizaje automático en la aplicación web

Una vez finalizado el proceso de entrenamiento y evaluación del modelo de aprendizaje automático en el entorno de Google Colab, se procede a su integración en una aplicación web con el objetivo de realizar predicciones en tiempo real. Para ello, el modelo entrenado es exportado al formato TensorFlow.js, el cual permite su ejecución directamente en el navegador sin necesidad de un servidor de procesamiento adicional.

El proceso de exportación genera un archivo principal denominado `model.json`, que contiene la arquitectura del modelo, así como uno o varios archivos con extensión `.bin`, los cuales almacenan los pesos aprendidos durante el entrenamiento. Estos archivos son indispensables para el correcto funcionamiento del sistema y deben ubicarse en el mismo directorio que la aplicación web o en una ruta accesible desde esta.



Dentro de la aplicación web, el modelo es incorporado mediante la librería TensorFlow.js, la cual se carga desde un repositorio externo. Una vez cargada la librería, se utiliza una función específica para leer el archivo `model.json`, lo que permite reconstruir el modelo completo en memoria y asociar automáticamente los pesos correspondientes. Este paso representa el momento en el cual el modelo queda disponible para su uso dentro de la aplicación.

```
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@2.0.0/dist/tf.min.js"></script>
```

```
// Carga del modelo
(async() => {
  console.log("Cargando modelo...");
  try {
    modelo = await tf.loadLayersModel("model.json");
    console.log("Modelo cargado");
    statusMsg.innerText = "Modelo Cargado. Iniciando Cámara...";
    statusMsg.style.color = "var(--neon-cyan)";
  } catch (error) {
    statusMsg.innerText = "Error cargando model.json";
    statusMsg.style.color = "red";
    console.error(error);
  }
})();
```

Debido a las restricciones de seguridad de los navegadores web, la aplicación no puede ejecutarse directamente desde el sistema de archivos local. Por esta razón, se requiere la utilización de un servidor web local que permita servir los archivos mediante el protocolo HTTP. Este procedimiento garantiza que el modelo pueda ser cargado correctamente y que la aplicación tenga acceso a los recursos necesarios.

```
PS C:\Users\carva\OneDrive\Documentos\Inteligencia Artificial\Segundo Parcial\EXAMEN> python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

Una vez cargado el modelo, la aplicación web accede a la cámara del dispositivo para capturar imágenes en tiempo real. Cada imagen obtenida es procesada siguiendo exactamente los mismos pasos aplicados durante el entrenamiento del modelo, incluyendo el redimensionamiento, la normalización de los valores de los píxeles y la conversión a escala de grises. Posteriormente, la imagen es transformada en un tensor con la forma requerida por el modelo.

```
function mostrarCamara() {  
    var opciones = {  
        audio: false,  
        video: {  
            facingMode: facingMode, width: tamano, height: tamano  
        }  
    }  
  
    if (navigator.mediaDevices.getUserMedia) {  
        navigator.mediaDevices.getUserMedia(opciones)  
            .then(function(stream) {  
                currentStream = stream;  
                video.srcObject = currentStream;  
                procesarCamara();  
                predecir();  
                statusMsg.innerText = "SISTEMA ACTIVO";  
            })  
            .catch(function(err) {  
                statusMsg.innerText = "Error de Cámara";  
                statusMsg.style.color = "red";  
                console.log(err);  
            })  
    } else {  
        alert("No existe la funcion getUserMedia");  
    }  
}
```

```
function procesarCamara() {  
    if (!currentStream) return;  
  
    // Dibujamos el video en el canvas visible  
    ctx.drawImage(video, 0, 0, tamano, tamano, 0, 0, tamano, tamano);  
  
    // Loop de dibujo  
    setTimeout(procesarCamara, 20);  
}
```

El tensor generado es enviado al modelo cargado para realizar la predicción correspondiente.

El resultado obtenido es un valor numérico que representa la probabilidad de pertenencia a

cada clase. En el caso de la clasificación binaria, este valor se interpreta mediante un umbral previamente definido, permitiendo determinar si la imagen corresponde a un perro o a un gato. El resultado final es mostrado en la interfaz de la aplicación web de forma clara y comprensible para el usuario.

```
var resultado = modelo.predict(tensor).dataSync();

// El resultado es un único valor (sigmoid). Asumimos 0 = gato, 1 = perro
var probabilidadPerro = resultado[0];
var probabilidadGato = 1 - probabilidadPerro;

// Convertir a porcentajes
var pctPerro = Math.round(probabilidadPerro * 100);
var pctGato = Math.round(probabilidadGato * 100);

// Actualizar textos
scoreDog.innerText = pctPerro + "%";
scoreCat.innerText = pctGato + "%";

// Actualizar anchos de barra
barDog.style.width = pctPerro + "%";
barCat.style.width = pctGato + "%";

// Actualizar Log si la confianza es alta
if (pctPerro > 80 && Math.random() > 0.95) {
  addLog("Detectado: PERRO (" + pctPerro + "%)", "log-dog");
} else if (pctGato > 80 && Math.random() > 0.95) {
  addLog("Detectado: GATO (" + pctGato + "%)", "log-cat");
}
```

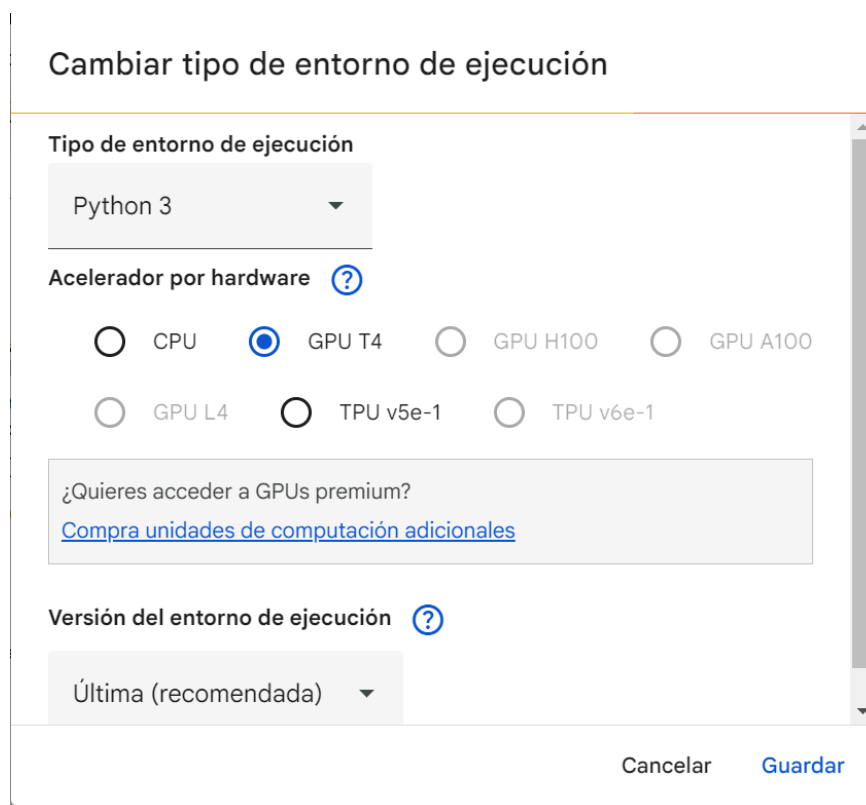
De esta manera, la integración del modelo en la aplicación web demuestra la viabilidad de utilizar modelos de aprendizaje automático entrenados en la nube para aplicaciones prácticas en tiempo real, facilitando la interacción entre el usuario y el sistema de reconocimiento de imágenes.

## Parte 2: Perros, Gatos y Pájaros

## 1. Configuración del entorno de trabajo

El desarrollo del proyecto se realiza en Google Colab, una plataforma en la nube que permite programar en Python sin necesidad de realizar instalaciones locales. Una de las principales ventajas de este entorno es la posibilidad de utilizar aceleración por GPU, lo cual reduce significativamente el tiempo de entrenamiento de los modelos de aprendizaje automático.

Antes de iniciar el desarrollo, se verifica que el tipo de entorno de ejecución esté configurado para utilizar GPU, ya que el entrenamiento de redes neuronales convolucionales con grandes volúmenes de imágenes puede resultar lento si se ejecuta únicamente en CPU. Esta configuración es un paso fundamental para optimizar el rendimiento del proceso de entrenamiento.



Cambiar tipo de entorno de ejecución

Tipo de entorno de ejecución  
Python 3 ▼

Acelerador por hardware ⓘ

☐ CPU ☒ GPU T4 ☐ GPU H100 ☐ GPU A100

☐ GPU L4 ☐ TPU v5e-1 ☐ TPU v6e-1

¿Quieres acceder a GPUs premium?  
[Compra unidades de computación adicionales](#)

Versión del entorno de ejecución ⓘ

Última (recomendada) ▼

Cancelar Guardar



## 2. Obtención del dataset

Para este entrenamiento, no se utilizó tensorflow datasets, debido a que en la librería no existe un dataset que cuente con tres animales. En su lugar, se buscó un dataset público en Kaggle, este dataset fue high-resolution-catdogbird-image-dataset-13000.

El proceso de descarga es distinto al dataset de catsvsdogs, ya que se debe configurar un API Key en Kaggle y configurarlo dentro de Google Colab para permitir la descarga dentro de los archivos de Colab.

En primer lugar se creó un JSON con la información respectiva al usuario y la API Key de Kaggle, este archivo se movió hacia la carpeta `/root/.kaggle/` y se asignó los permisos respectivos para su correcto funcionamiento.

```
%%writefile kaggle.json
{
  "username": "cesargoat",
  "key": "e2bc5f196b92b82186d810f2299f56e2"
}

Writing kaggle.json

!mkdir -p /root/.kaggle
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

Una vez configurado Kaggle, se procede a verificar que este correctamente instalado listando ocho datasets, una vez verificado, se descarga el dataset para el proyecto de gatos perros y pájaros el cual contiene aproximadamente 13 000 imágenes.

```
[3]
✓ 0 s
!kaggle datasets list | head

ref                                     title                                     size  lastUpdate
-----
wardabilal/spotify-global-music-dataset-20092025  Spotify Global Music Dataset (2009-2025)  1289021  2025-11-11
neurocipher/heartdisease                    Heart Disease                               3491    2025-12-11
kundanbedmutha/exam-score-prediction-dataset    Exam Score Prediction Dataset              325454  2025-11-21
neurocipher/student-performance              Student Performance                         49705   2025-12-11
aliiihussain/spotify-analysis-and-visualization  Spotify Analysis & Visualization          2369    2025-12-01
hetmengar/ola-and-uber-ride-booking-and-cancellation-data  Ola & Uber: Ride Booking & Cancellation Data  3678531  2025-11-21
khushikyad001/ai-impact-on-jobs-2030          AI Impact on Jobs 2030                    87410   2025-11-01
rohiteng/amazon-sales-dataset                Amazon Sales Dataset                       4037578  2025-11-21

[4]
✓ 10 m
# Descargar el nuevo dataset
!kaggle datasets download -d mahmoudnoor/high-resolution-catdogbird-image-dataset-13000

# Crear una carpeta llamada 'dataset' y descomprimir allí
!mkdir -p dataset
!unzip -q high-resolution-catdogbird-image-dataset-13000.zip -d dataset

Dataset URL: https://www.kaggle.com/datasets/mahmoudnoor/high-resolution-catdogbird-image-dataset-13000
License(s): copyright-authors
high-resolution-catdogbird-image-dataset-13000.zip: Skipping, found more recently modified local copy (use --force to force download)
```

Posteriormente, se cargó el dataset de entrenamiento y de validación en dos variables, train\_ds y val\_ds respectivamente. Para verificar que se agregó correctamente se imprime las cabeceras del dataset.

```
import tensorflow as tf
import os

# Apuntar a la nueva carpeta donde descomprimos los datos
data_dir = "dataset"

# Cargar dataset de entrenamiento
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)

# Cargar dataset de validación
val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)

# Verificar las nuevas clases encontradas (debería mostrar ['birds', 'cat', 'dog'])
print("Clases encontradas:", train_ds.class_names)
```

```
** Found 13344 files belonging to 3 classes.
Using 10676 files for training.
Found 13344 files belonging to 3 classes.
Using 2668 files for validation.
Clases encontradas: ['bird', 'cat', 'dog']
```

Tanto la variable `train_ds` y `val_ds` forman parte de los datos de entrenamiento, por ello, se creó una única variable llamada `datos` que contenga a estas dos variables.

```
#Crear variable datos
datos = {
    'train': train_ds,
    'validation': val_ds
}
```

En esta fase se realizó una visualización preliminar de los datos de entrenamiento con el fin de verificar su correcta carga y etiquetado. Para ello, se emplearon ciclos repetitivos para imprimir cada imagen y cada título de la imagen cinco veces.



## Preprocesamiento de imágenes

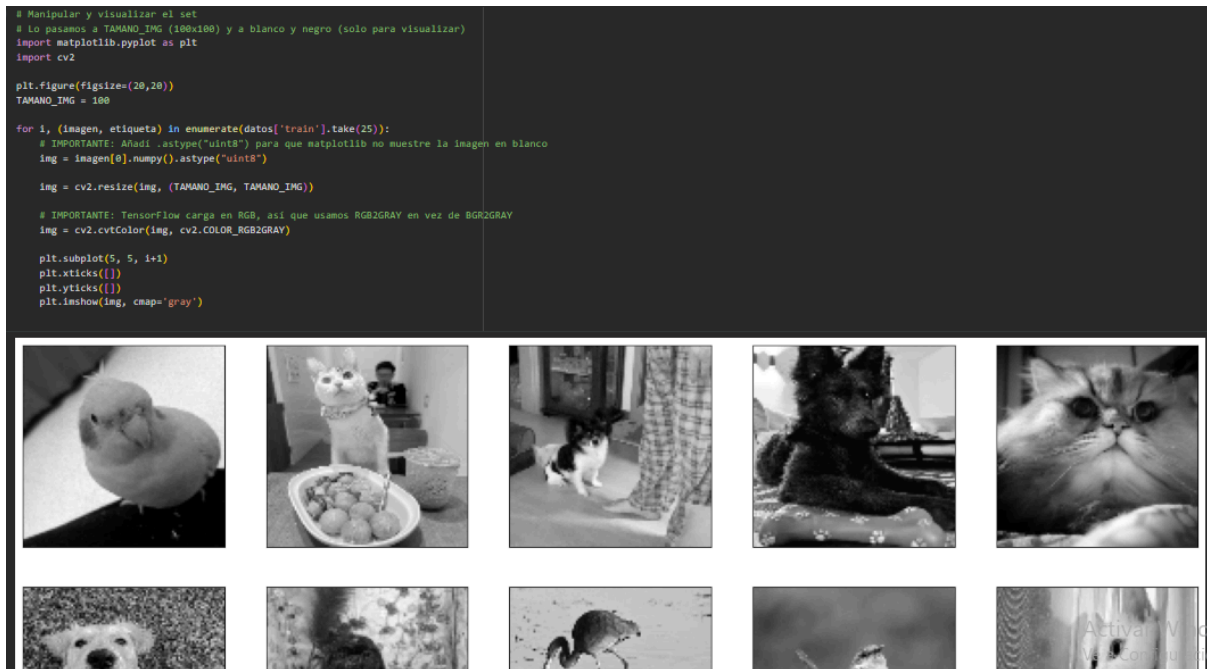
El preprocesamiento es una etapa clave para garantizar un entrenamiento adecuado del modelo.

### Redimensionamiento

Todas las imágenes son redimensionadas a un tamaño fijo de  $100 \times 100$  píxeles. Este tamaño representa un equilibrio entre conservar información visual relevante y reducir el costo computacional del entrenamiento.

### Conversión a escala de grises

Las imágenes son convertidas a escala de grises, reduciendo el número de canales de color de tres (RGB) a uno. Esta decisión permite disminuir la complejidad del modelo y acelerar el entrenamiento, manteniendo suficiente información visual para distinguir entre las clases.



## Ajuste de forma

Cada imagen se reorganiza para tener la forma (100, 100, 1), donde el último valor indica un único canal de color. Esta estructura es necesaria para que TensorFlow procese correctamente las imágenes dentro de la red neuronal. Posteriormente, cada imagen procesada se almacenó junto con su etiqueta asociada en una lista destinada a los datos de entrenamiento.

```
#Variable que contendrá todos los pares de los datos (imagen y etiqueta) ya modificados (blanco y negro, 100x100)
datos_entrenamiento = []

for imagen_batch, etiqueta_batch in datos['train']: # todos los datos
    for i in range(imagen_batch.shape[0]):
        imagen = imagen_batch[i].numpy()
        |      etiqueta = etiqueta_batch[i].numpy()

        imagen = cv2.resize(imagen, (TAMANO_IMG, TAMANO_IMG))
        imagen = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
        imagen = imagen.reshape(TAMANO_IMG, TAMANO_IMG, 1)

        datos_entrenamiento.append([imagen, etiqueta])
```

## Separación de variables

Los datos se separan en dos conjuntos:

- X: contiene las imágenes procesadas en forma de matrices de píxeles.
- Y: contiene las respuestas dadas por el modelo (gato, perro o pájaro)

```
#Preparar mis variables X (entradas) y y (etiquetas) separadas

X = [] #imagenes de entrada (píxeles)
y = [] #etiquetas (perro gato o pájaro)

for imagen, etiqueta in datos_entrenamiento:
    X.append(imagen)
    y.append(etiqueta)
```

## Normalización

Los valores de los píxeles, originalmente comprendidos entre 0 y 255, se normalizan a un rango de 0 a 1 dividiendo cada valor entre 255. Este paso mejora la estabilidad numérica del entrenamiento y facilita la convergencia del modelo.

```
#Normalizar los datos de las X (imagenes). Se pasan a float
import numpy as np

X = np.array(X).astype(float) / 255
```

## Definición de modelos de entrenamiento

Se diseñan y entrenan tres modelos distintos, con el objetivo de comparar su desempeño y analizar sus ventajas y desventajas. A diferencia del modelo de perros y gatos, este modelo se entrena con Softmax al tener tres posibles resultados, esta configuración devolverá la probabilidad de cada uno, sumando las tres probabilidades resultará el 100%.

### Modelo 1: Red neuronal densa

Este modelo utiliza únicamente capas densas:

- Recibe como entrada imágenes de  $100 \times 100$  píxeles en escala de grises, 1 canal de color..
- Utiliza una capa de aplanamiento (Flatten) para convertir la imagen en un vector lineal de datos.
- Dos capas densas intermedias con aproximadamente 150 neuronas cada una con activación ReLU para procesar la información.
- Una capa de salida con tres neuronas (una por cada animal) y función de activación softmax, permitiendo la clasificación multiclase.

Este modelo sirve como referencia inicial, aunque suele ser menos efectivo que las redes convolucionales para reconocer patrones visuales complejos.

### Modelo 2: Red neuronal convolucional (CNN)

Este modelo incorpora capas convolucionales y de agrupamiento máximo (max pooling):

- Emplea tres bloques de convolución y *max pooling* con 32, 64 y 128 filtros respectivamente.
- Varias capas convolucionales con un número creciente de filtros.
- Capas de reducción de dimensionalidad mediante pooling.
- Una capa densa final con aproximadamente 100 neuronas.
- Capa de salida con activación softmax, necesaria para la clasificación de 3 clases.

Este tipo de red es más adecuada para el reconocimiento de patrones visuales y suele reportar mejores resultados que las redes simples.

### **Modelo 3: CNN con Dropout**

Este modelo es similar al anterior, pero incluye una capa Dropout con una tasa de 0.5 antes de la capa densa final y utiliza una capa densa de 250 neuronas antes de la capa de salida. El objetivo de esta técnica es reducir el sobreajuste al desactivar aleatoriamente neuronas durante el entrenamiento.



```
# Crear los modelos iniciales
# Usamos SOFTMAX como salida porque tenemos 3 opciones (Gato, Perro, Pájaro).
# Softmax nos dará la probabilidad de cada uno, y la suma de las 3 dará 1 (100%).

modeloDenso = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(100, 100, 1)), # El 1 es porque es blanco y negro
    tf.keras.layers.Dense(150, activation='relu'),
    tf.keras.layers.Dense(150, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax') # 3 Neuronas de salida
])

modeloCNN = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(100, 100, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax') # 3 Neuronas de salida
])

modeloCNN2 = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(100, 100, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),

    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(250, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax') # 3 Neuronas de salida
])
```

## Entrenamiento y evaluación inicial (sin aumento de datos)

Los modelos se entrenan utilizando:

- Optimizador Adam: Seleccionado por su excelente desempeño general y capacidad para ajustar la tasa de aprendizaje automáticamente
- Función de pérdida Sparse Categorical Crossentropy: Utilizada en lugar de la binaria para clasificar múltiples opciones.
- Métrica accuracy: para medir el porcentaje de predicciones correctas durante el entrenamiento y la validación.

Se separa un porcentaje de los datos para validación y se monitorea el entrenamiento mediante TensorBoard, lo que permite analizar gráficas de precisión y pérdida.

```
# Compilar modelos (clasificación multiclase: gato, perro, pájaro)

modeloDenso.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

modeloCNN.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

modeloCNN2.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

### Entrenamiento del modelo denso y registro con TensorBoard

En esta fase se realizó el entrenamiento del modelo de red neuronal densa utilizando los datos previamente preparados. Para el seguimiento del proceso de aprendizaje se empleó la herramienta TensorBoard, la cual permite registrar y visualizar métricas de entrenamiento a lo largo de las épocas.

### Configuración del entrenamiento

- Se utilizó un tamaño de lote (batch size) de 32 imágenes.
- Se definió un total de 100 épocas de entrenamiento.
- Se reservó automáticamente un 15 % de los datos para validación.

- Se incorporó TensorBoard mediante el arreglo de callbacks.

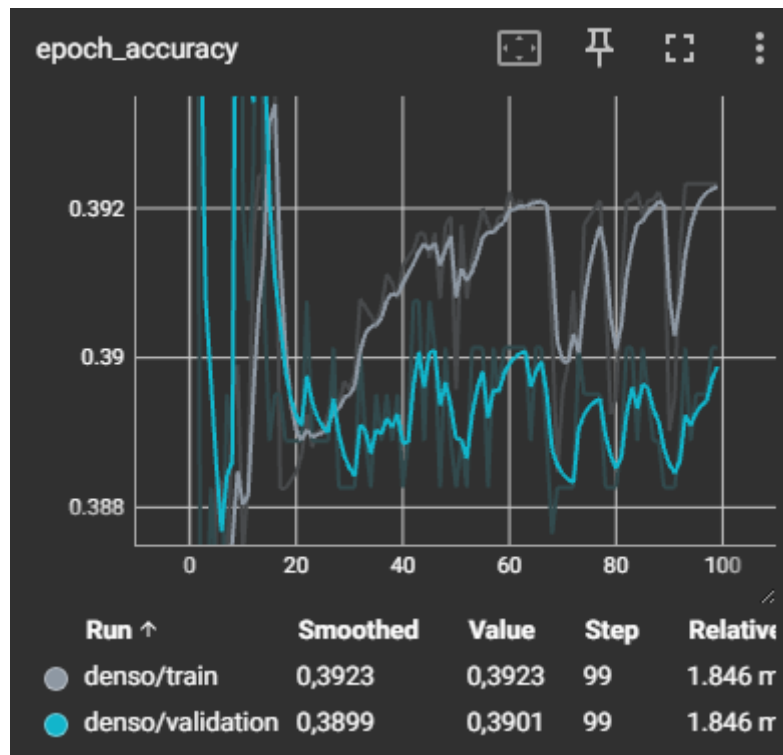
### **Uso de TensorBoard**

- Los resultados de cada época se almacenaron en un directorio específico.
- Se registraron métricas como precisión (accuracy) y función de pérdida (loss).
- Esta información permite generar gráficas para analizar el comportamiento del modelo.

### **Análisis del desempeño del modelo denso**

#### **Precisión (Accuracy)**

- La precisión del conjunto de entrenamiento aumenta de forma constante a lo largo de las épocas, alcanzando valores cercanos al 48–49 %.
- La precisión del conjunto de validación se mantiene significativamente más baja, alrededor del 39–40 %, sin mostrar una mejora sostenida.
- La divergencia creciente entre ambas curvas indica que el modelo aprende los datos de entrenamiento, pero no generaliza adecuadamente.



### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye de manera progresiva, lo cual sugiere que el modelo optimiza correctamente sobre los datos que ve durante el aprendizaje.
- En contraste, la pérdida del conjunto de validación aumenta conforme avanzan las épocas.
- Este comportamiento evidencia que el modelo empeora su desempeño sobre datos no vistos.



## Entrenamiento de la CNN

Durante el entrenamiento del modelo CNN se observa un aumento progresivo de la precisión y una disminución de la pérdida en el conjunto de entrenamiento. En las primeras épocas, el conjunto de validación muestra un comportamiento similar, indicando una correcta capacidad de aprendizaje inicial.

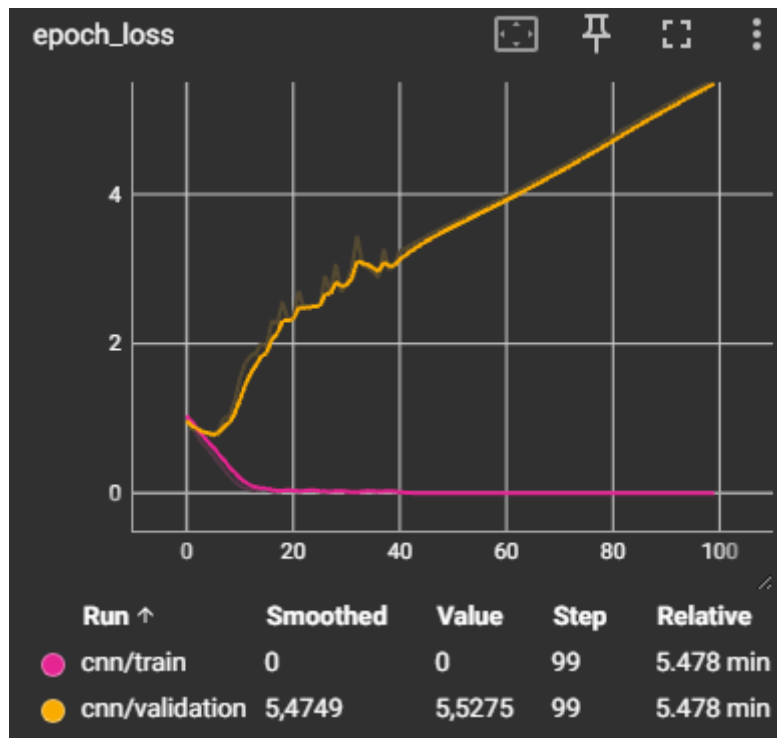
```
tensorboardCNN = TensorBoard(log_dir='logs/cnn')
modeloCNN.fit(X, y, batch_size=32,
              validation_split=0.15,
              epochs=100,
              callbacks=[tensorboardCNN])
```

Epoch 1/100  
284/284 [=====] - 7s 15ms/step - loss: 1.0387 - accuracy: 0.4557 - val\_loss: 1.0023 - val\_accuracy: 0.4838  
Epoch 2/100  
284/284 [=====] - 3s 11ms/step - loss: 0.9097 - accuracy: 0.5569 - val\_loss: 0.8341 - val\_accuracy: 0.6042  
Epoch 3/100  
284/284 [=====] - 3s 12ms/step - loss: 0.7699 - accuracy: 0.6504 - val\_loss: 0.8336 - val\_accuracy: 0.6186  
Epoch 4/100  
284/284 [=====] - 3s 12ms/step - loss: 0.6772 - accuracy: 0.6985 - val\_loss: 0.7979 - val\_accuracy: 0.6417  
Epoch 5/100  
284/284 [=====] - 4s 13ms/step - loss: 0.6001 - accuracy: 0.7389 - val\_loss: 0.7849 - val\_accuracy: 0.6429  
Epoch 6/100  
284/284 [=====] - 3s 11ms/step - loss: 0.5124 - accuracy: 0.7825 - val\_loss: 0.8802 - val\_accuracy: 0.6305  
Epoch 7/100  
284/284 [=====] - 3s 11ms/step - loss: 0.4156 - accuracy: 0.8277 - val\_loss: 0.9703 - val\_accuracy: 0.6248  
Epoch 8/100  
284/284 [=====] - 3s 12ms/step - loss: 0.3200 - accuracy: 0.8707 - val\_loss: 0.9820 - val\_accuracy: 0.6498  
Epoch 9/100  
284/284 [=====] - 3s 12ms/step - loss: 0.2157 - accuracy: 0.9192 - val\_loss: 1.1174 - val\_accuracy: 0.6461  
Epoch 10/100

## Análisis del desempeño del modelo convolucional

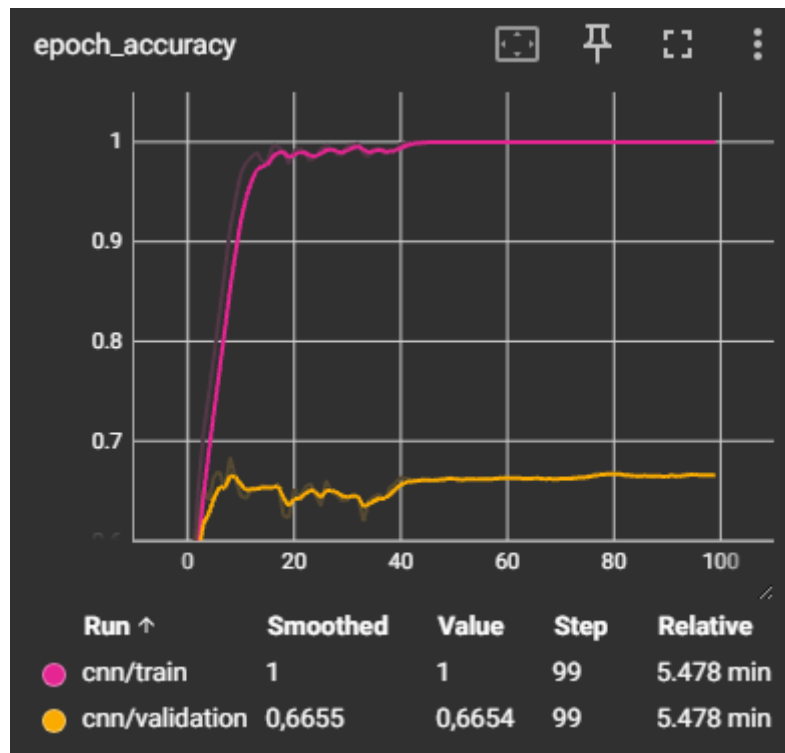
### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye rápidamente hasta valores cercanos a cero.
- En contraste, la pérdida del conjunto de validación aumenta de forma progresiva, alcanzando valores elevados al final del entrenamiento.
- Esta divergencia indica que el modelo aprende muy bien los datos de entrenamiento, pero presenta un deterioro significativo al evaluar datos no vistos.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento alcanza valores cercanos al 99 %, lo que evidencia una alta capacidad de ajuste sobre los datos de entrenamiento.
- La precisión del conjunto de validación se mantiene alrededor del 64 %, mostrando un desempeño considerablemente inferior.
- La brecha constante entre ambas curvas confirma la falta de generalización del modelo.



### Interpretación del comportamiento

- El modelo convolucional extrae características relevantes de las imágenes con gran eficacia.
- Sin embargo, el rápido descenso de la pérdida de entrenamiento y el aumento simultáneo de la pérdida de validación son indicativos de sobreajuste severo.
- El modelo memoriza patrones específicos del conjunto de entrenamiento en lugar de aprender representaciones generalizables.

Aunque el modelo convolucional logra una alta precisión en el conjunto de entrenamiento y mejora notablemente el desempeño respecto al modelo denso, los resultados evidencian un sobreajuste pronunciado. Esto resalta la necesidad de incorporar técnicas adicionales como



aumento de datos, regularización más agresiva o una reducción en la complejidad del modelo para mejorar su capacidad de generalización.

## Entrenamiento del modelo CNN2

El modelo CNN2 muestra un aprendizaje progresivo tanto en el conjunto de entrenamiento como en el de validación durante las primeras épocas. La precisión de validación alcanza valores cercanos al 85% mientras la pérdida disminuye, lo que indica una mejor capacidad de generalización inicial en comparación con el modelo anterior.

```
tensorboardCNN2 = TensorBoard(log_dir='logs/cnn2')
modeloCNN2.fit(X, y, batch_size=32,
               validation_split=0.15,
               epochs=100,
               callbacks=[tensorboardCNN2])
```

| Epoch        | loss   | accuracy | val_loss | val_accuracy |
|--------------|--------|----------|----------|--------------|
| 284/284      | 0.0253 | 0.9915   | 2.1430   | 0.6579       |
| Epoch 73/100 |        |          |          |              |
| 284/284      | 0.0225 | 0.9931   | 2.1826   | 0.6586       |
| Epoch 74/100 |        |          |          |              |
| 284/284      | 0.0263 | 0.9928   | 1.9870   | 0.6723       |
| Epoch 75/100 |        |          |          |              |
| 284/284      | 0.0329 | 0.9902   | 2.1694   | 0.6504       |
| Epoch 76/100 |        |          |          |              |
| 284/284      | 0.0246 | 0.9917   | 1.9424   | 0.6523       |
| Epoch 77/100 |        |          |          |              |
| 284/284      | 0.0193 | 0.9926   | 2.0991   | 0.6411       |
| Epoch 78/100 |        |          |          |              |
| 284/284      | 0.0265 | 0.9921   | 2.2806   | 0.6604       |
| Epoch 79/100 |        |          |          |              |
| 284/284      | 0.0237 | 0.9934   | 2.2592   | 0.6548       |
| Epoch 80/100 |        |          |          |              |
| 284/284      | 0.0288 | 0.9913   | 2.1678   | 0.6567       |
| Epoch 81/100 |        |          |          |              |
| 284/284      | 0.0282 | 0.9900   | 2.1711   | 0.6592       |
| Epoch 82/100 |        |          |          |              |

## Análisis del desempeño del modelo convolucional con regularización

### Función de pérdida (Loss)

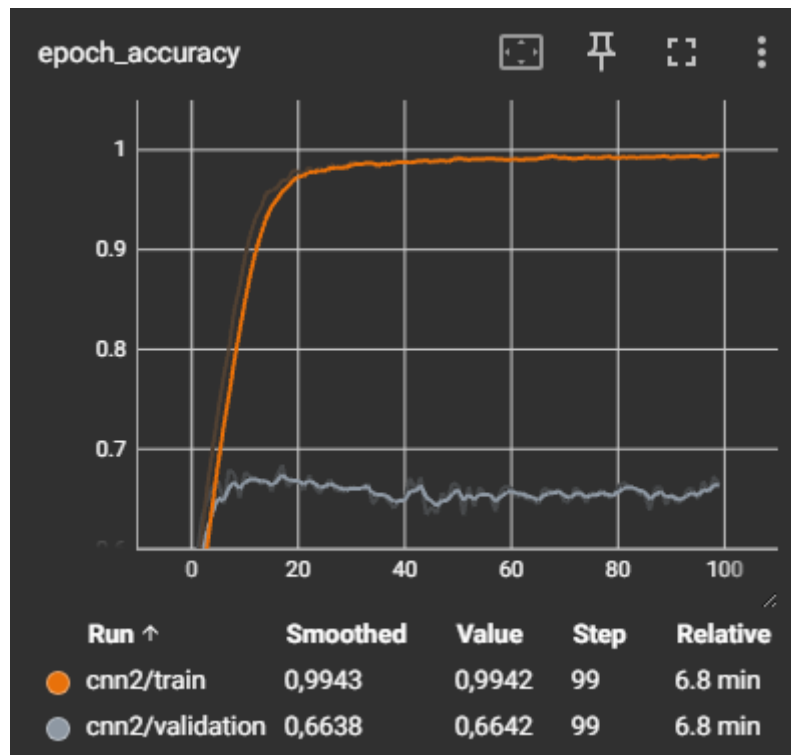
- La pérdida del conjunto de entrenamiento disminuye de manera sostenida hasta valores cercanos a cero 2% aproximadamente.
- La pérdida del conjunto de validación muestra una tendencia creciente, aunque con valores máximos inferiores a los observados en el modelo convolucional sin regularización.

- Este comportamiento indica que el dropout atenúa parcialmente el sobreajuste, aunque no logra eliminarlo por completo.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento alcanza valores cercanos al 99.2 %, lo que refleja una alta capacidad de ajuste del modelo.
- La precisión del conjunto de validación se estabiliza alrededor del 66.4 %, superando el desempeño del modelo convolucional sin dropout.
- La brecha entre entrenamiento y validación es menor, lo que sugiere una mejora en la generalización.



### Interpretación del comportamiento

- La incorporación de dropout reduce la dependencia excesiva de neuronas individuales durante el entrenamiento.
- El modelo mantiene una alta capacidad de aprendizaje sin sacrificar significativamente el desempeño en datos no vistos.
- A pesar de la mejora, persisten indicios de sobreajuste, evidenciados por la diferencia entre las curvas de pérdida.

El modelo convolucional con dropout presenta un mejor equilibrio entre aprendizaje y generalización en comparación con las arquitecturas previas. Si bien no elimina completamente el sobreajuste, constituye una mejora significativa y demuestra la efectividad

de la regularización. Estos resultados respaldan la incorporación de técnicas adicionales, como el aumento de datos, para optimizar aún más el desempeño del modelo.

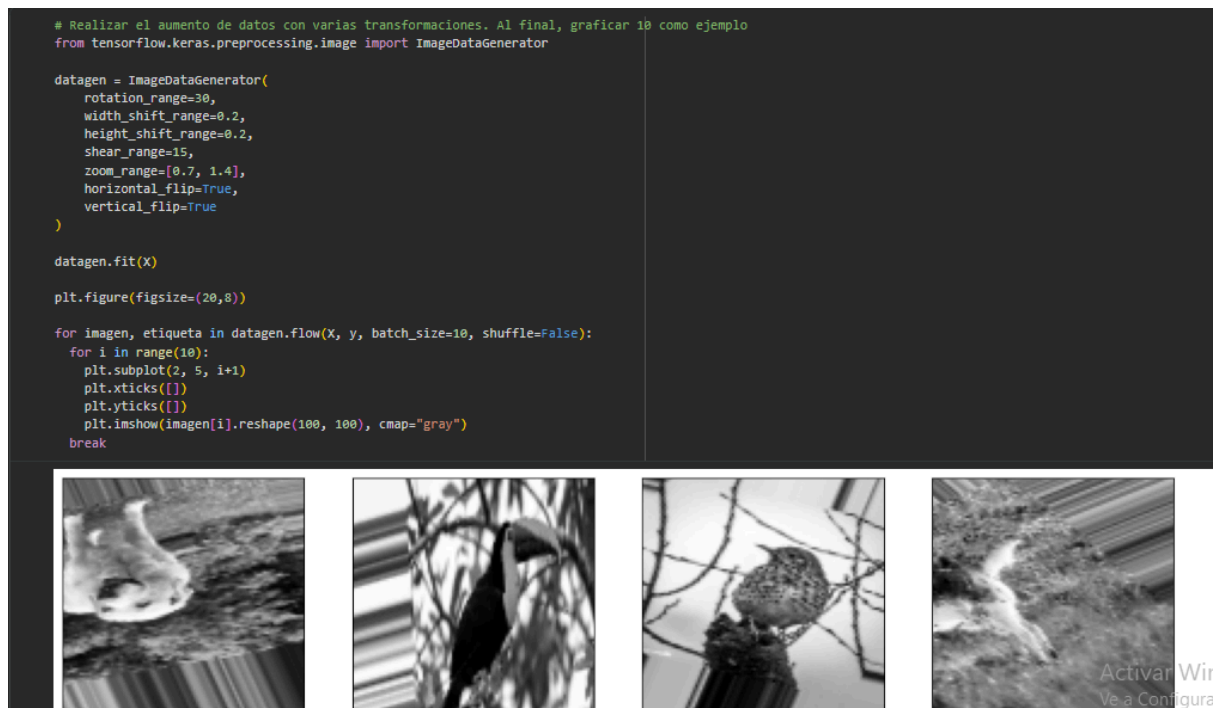
### **Aplicación de aumento de datos (Data Augmentation)**

En esta etapa se aplicó la técnica de aumento de datos con el objetivo de incrementar la variabilidad del conjunto de entrenamiento sin necesidad de recolectar nuevas imágenes. Para ello, se utilizó la clase ImageDataGenerator de Keras, la cual permite generar transformaciones aleatorias sobre las imágenes originales durante el entrenamiento.

### **Transformaciones aplicadas**

Se configuraron las siguientes operaciones de aumento de datos:

- Rotación aleatoria de hasta 30 grados.
- Desplazamientos horizontales y verticales de hasta el 20 %.
- Transformaciones de cizallamiento (shear).
- Escalado aleatorio (zoom in y zoom out).
- Inversión horizontal y vertical de las imágenes..



Este proceso permite simular una mayor diversidad de datos sin necesidad de recolectar nuevas imágenes.

### Definición de modelos con aumento de datos

En esta etapa se definieron nuevamente tres arquitecturas de redes neuronales, las cuales serán entrenadas utilizando aumento de datos. Las estructuras de los modelos se mantienen iguales a las versiones previas, con el fin de evaluar exclusivamente el impacto del aumento de datos en el desempeño y la generalización.

### Modelo denso con aumento de datos

- Recibe imágenes de entrada de  $100 \times 100$  píxeles en escala de grises.
- Utiliza una capa de aplanamiento (Flatten).
- Incluye dos capas densas de 150 neuronas con activación ReLU.

- La capa de salida contiene una neurona con activación softmax para clasificación en varias categorías

```
tensorboardDensio_AD = TensorBoard(log_dir='logs/densio_AD')

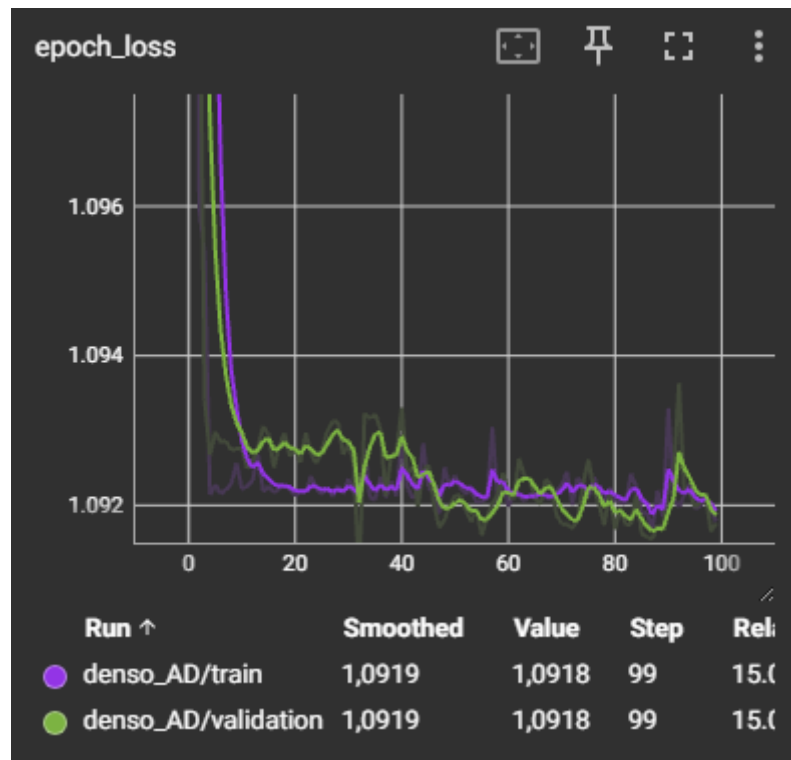
modeloDensio_AD.fit(
    data_gen_entrenamiento,
    epochs=100, batch_size=32,
    validation_data=(X_validacion, y_validacion),
    steps_per_epoch=int(np.ceil(len(X_entrenamiento) / float(32))),
    validation_steps=int(np.ceil(len(X_validacion) / float(32))),
    callbacks=[tensorboardDensio_AD]
)

Epoch 1/100
284/284 [=====] - 11s 32ms/step - loss: 1.2270 - accuracy: 0.3553 - val_loss: 1.1315 - val_accuracy: 0.3065
Epoch 2/100
284/284 [=====] - 10s 34ms/step - loss: 1.1032 - accuracy: 0.3684 - val_loss: 1.0912 - val_accuracy: 0.3951
Epoch 3/100
284/284 [=====] - 10s 34ms/step - loss: 1.0926 - accuracy: 0.3894 - val_loss: 1.0887 - val_accuracy: 0.3876
Epoch 4/100
284/284 [=====] - 10s 35ms/step - loss: 1.0934 - accuracy: 0.3849 - val_loss: 1.0924 - val_accuracy: 0.3864
Epoch 5/100
284/284 [=====] - 9s 31ms/step - loss: 1.0921 - accuracy: 0.3876 - val_loss: 1.0935 - val_accuracy: 0.3864
Epoch 6/100
284/284 [=====] - 10s 34ms/step - loss: 1.0922 - accuracy: 0.3877 - val_loss: 1.0932 - val_accuracy: 0.3864
Epoch 7/100
284/284 [=====] - 10s 35ms/step - loss: 1.0923 - accuracy: 0.3877 - val_loss: 1.0934 - val_accuracy: 0.3864
Epoch 8/100
284/284 [=====] - 10s 35ms/step - loss: 1.0922 - accuracy: 0.3877 - val_loss: 1.0924 - val_accuracy: 0.3864
Epoch 9/100
284/284 [=====] - 9s 31ms/step - loss: 1.0923 - accuracy: 0.3877 - val_loss: 1.0929 - val_accuracy: 0.3864
Epoch 10/100
284/284 [=====] - 10s 35ms/step - loss: 1.0923 - accuracy: 0.3877 - val_loss: 1.0930 - val_accuracy: 0.3864
Epoch 11/100
284/284 [=====] - 10s 35ms/step - loss: 1.0922 - accuracy: 0.3877 - val_loss: 1.0930 - val_accuracy: 0.3864
```

## Análisis del desempeño del modelo denso

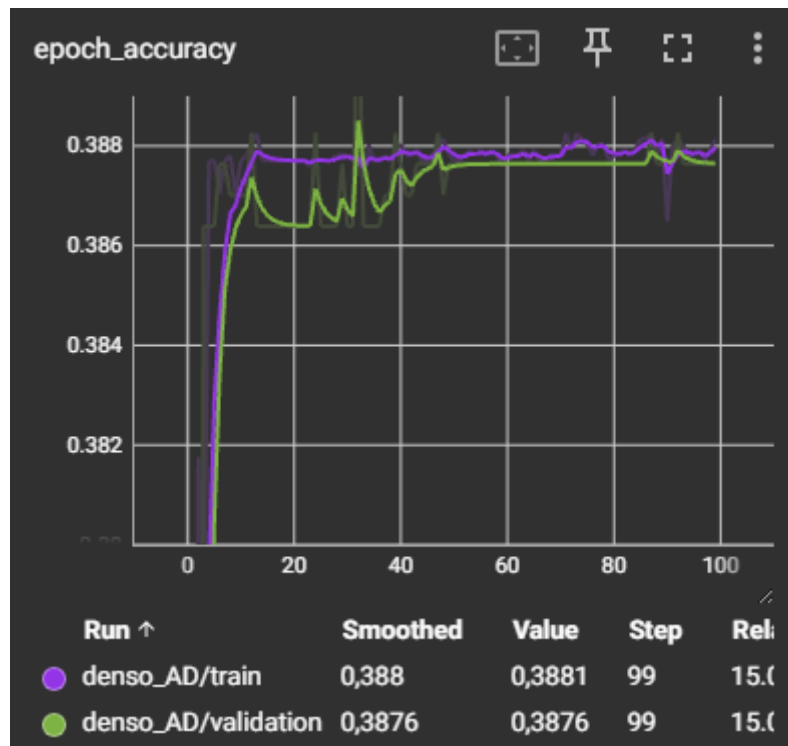
### Función de pérdida (Loss)

- La función de pérdida del conjunto de entrenamiento se mantiene estancada en valores altos cercanos a 1.092 a lo largo de las épocas, sin mostrar un descenso significativo.
- La pérdida de validación oscila alrededor de 1.093.
- Este comportamiento, donde la pérdida no disminuye, indica que el modelo no está logrando aprender patrones efectivos para diferenciar entre las tres clases, comportándose de manera casi aleatoria.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento apenas alcanza el 38.8%, un valor muy bajo para una tarea de clasificación de tres clases.
- La precisión del conjunto de validación se mantiene en cifras similares, rondando el 38.7%, sin mostrar ninguna mejora significativa a lo largo del tiempo.
- Dado que tenemos 3 clases, una elección puramente al azar daría un 33.3%; el hecho de que el modelo apenas supere este umbral confirma que la arquitectura densa no es capaz de procesar la variabilidad introducida por el aumento de datos."



### Modelo convolucional con aumento de datos

- Emplea tres capas convolucionales con 32, 64 y 128 filtros respectivamente.
- Cada capa convolucional es seguida por una capa de max pooling para la reducción espacial.
- Finaliza con una capa densa de 100 neuronas y una capa de salida softmax para diferenciar tres categorías.



```
tensorboardCNN_AD = TensorBoard(log_dir='logs/cnn_AD')

modeloCNN_AD.fit(
    data_gen_entrenamiento,
    epochs=150, batch_size=32,
    validation_data=(X_validacion, y_validacion),
    steps_per_epoch=int(np.ceil(len(X_entrenamiento) / float(32))),
    validation_steps=int(np.ceil(len(X_validacion) / float(32))),
    callbacks=[tensorboardCNN_AD]
)

284/284 [=====] - 11s 40ms/step - loss: 0.4947 - accuracy: 0.7915 - val_loss: 0.5815 - val_accuracy: 0.7665
Epoch 123/150
284/284 [=====] - 11s 40ms/step - loss: 0.4893 - accuracy: 0.7957 - val_loss: 0.5780 - val_accuracy: 0.7697
Epoch 124/150
284/284 [=====] - 11s 40ms/step - loss: 0.4974 - accuracy: 0.7887 - val_loss: 0.6342 - val_accuracy: 0.7547
Epoch 125/150
284/284 [=====] - 11s 39ms/step - loss: 0.4929 - accuracy: 0.7909 - val_loss: 0.5793 - val_accuracy: 0.7647
Epoch 126/150
284/284 [=====] - 11s 39ms/step - loss: 0.4821 - accuracy: 0.7976 - val_loss: 0.5380 - val_accuracy: 0.7828
Epoch 127/150
284/284 [=====] - 11s 40ms/step - loss: 0.4909 - accuracy: 0.7913 - val_loss: 0.5616 - val_accuracy: 0.7790
Epoch 128/150
284/284 [=====] - 10s 37ms/step - loss: 0.5054 - accuracy: 0.7886 - val_loss: 0.5832 - val_accuracy: 0.7659
Epoch 129/150
284/284 [=====] - 11s 38ms/step - loss: 0.4831 - accuracy: 0.7987 - val_loss: 0.5731 - val_accuracy: 0.7809
Epoch 130/150
284/284 [=====] - 11s 39ms/step - loss: 0.4873 - accuracy: 0.7918 - val_loss: 0.5593 - val_accuracy: 0.7753
Epoch 131/150
284/284 [=====] - 11s 39ms/step - loss: 0.4849 - accuracy: 0.7995 - val_loss: 0.6155 - val_accuracy: 0.7516
Epoch 132/150
284/284 [=====] - 11s 39ms/step - loss: 0.4903 - accuracy: 0.7958 - val_loss: 0.5171 - val_accuracy: 0.7884
Epoch 133/150
```

## Análisis del desempeño del modelo de red neuronal convolucional con aumento de datos

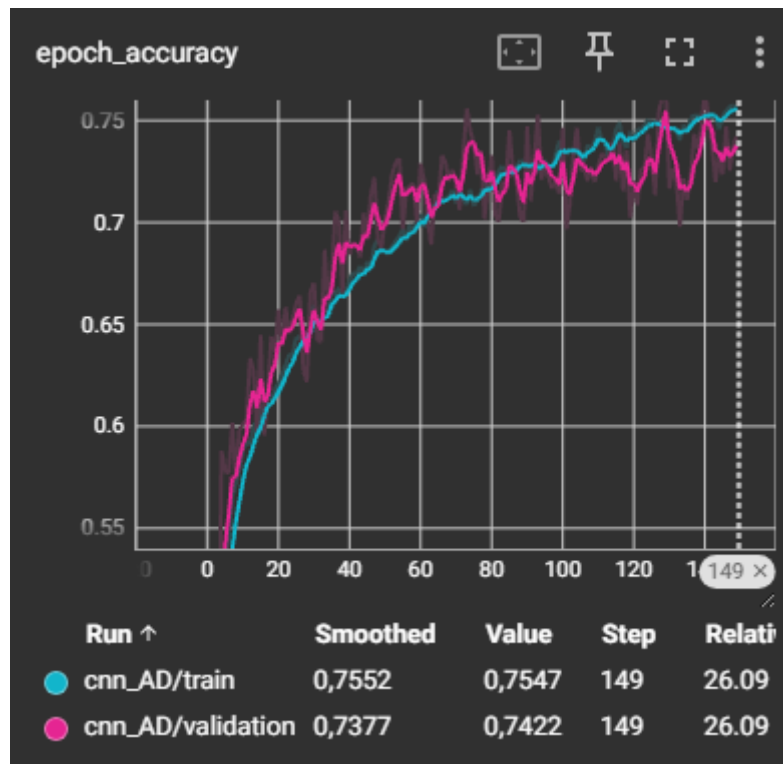
### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento disminuye de forma constante y saludable, descendiendo hasta valores cercanos a 0.48 al final del proceso.
- La pérdida del conjunto de validación acompaña esta tendencia descendente llegando a 0.58, aunque se mantiene ligeramente por encima de la curva de entrenamiento.
- La brecha entre ambas curvas es mucho menor en comparación con los modelos sin aumento de datos, lo que sugiere que la variabilidad extra ha ayudado a regularizar el modelo.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento aumenta progresivamente hasta alcanzar valores cercanos al 80%, evidenciando una buena capacidad de ajuste.
- La precisión del conjunto de validación alcanza un 76.6%, mostrando un desempeño sólido y consistente con los datos de entrenamiento.
- Aunque persiste una pequeña diferencia entre entrenamiento y validación, el modelo demuestra una capacidad de generalización muy superior a la arquitectura densa.



### Interpretación del comportamiento

- El aumento de datos aporta diversidad suficiente para evitar la memorización del conjunto de entrenamiento.
- El modelo logra aprender representaciones robustas de las imágenes sin necesidad de una regularización adicional agresiva.
- La arquitectura convolucional resulta adecuada para aprovechar las transformaciones aplicadas a los datos.

El modelo convolucional con aumento de datos presenta un desempeño sólido y equilibrado, con alta precisión y pérdidas decrecientes tanto en entrenamiento como en validación. Estos resultados confirman que el aumento de datos, combinado con redes convolucionales, es una estrategia efectiva para mejorar la generalización en problemas de clasificación de imágenes.

## Modelo convolucional con dropout y aumento de datos

- Presenta una arquitectura similar al modelo convolucional estándar.
- Incorpora una capa de dropout con una tasa de 0.5 antes de las capas densas.
- Utiliza una capa densa de 250 neuronas para aumentar la capacidad de representación.
- La capa de salida emplea activación softmax para diferenciar varias categorías.

```
tensorboardCNN2_AD = TensorBoard(log_dir='logs/cnn2_AD')

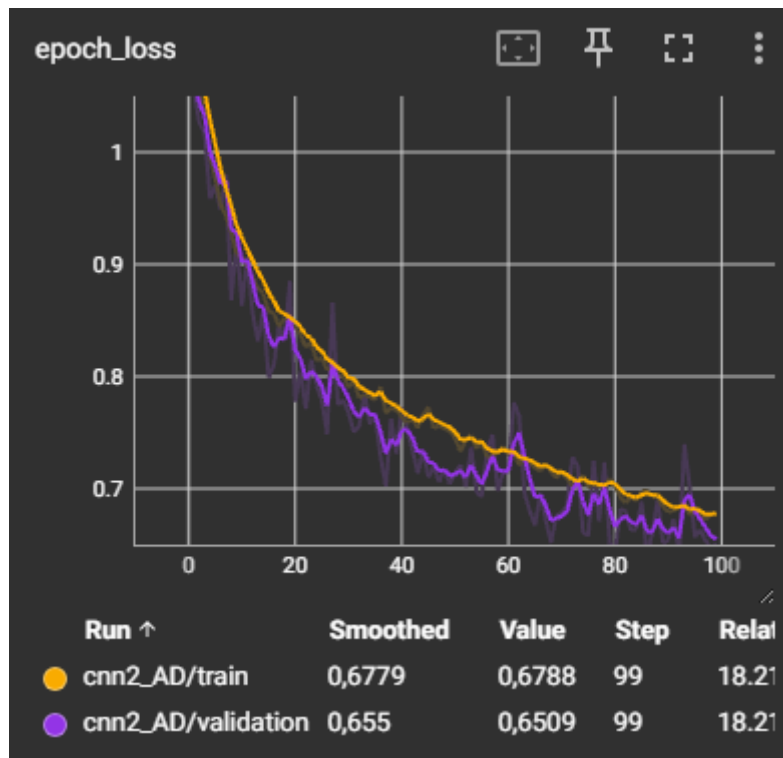
modeloCNN2_AD.fit(
    data_gen_entrenamiento,
    epochs=100, batch_size=32,
    validation_data=(X_validacion, y_validacion),
    steps_per_epoch=int(np.ceil(len(X_entrenamiento) / float(32))),
    validation_steps=int(np.ceil(len(X_validacion) / float(32))),
    callbacks=[tensorboardCNN2_AD]
)

Epoch 1/100
284/284 [=====] - 14s 41ms/step - loss: 1.0936 - accuracy: 0.3929 - val_loss: 1.0752 - val_accuracy: 0.4220
Epoch 2/100
284/284 [=====] - 11s 40ms/step - loss: 1.0740 - accuracy: 0.4154 - val_loss: 1.0603 - val_accuracy: 0.4182
Epoch 3/100
284/284 [=====] - 11s 40ms/step - loss: 1.0415 - accuracy: 0.4505 - val_loss: 1.0224 - val_accuracy: 0.4844
Epoch 4/100
284/284 [=====] - 11s 40ms/step - loss: 1.0131 - accuracy: 0.4717 - val_loss: 0.9883 - val_accuracy: 0.5025
Epoch 5/100
284/284 [=====] - 11s 40ms/step - loss: 1.0001 - accuracy: 0.4858 - val_loss: 0.9423 - val_accuracy: 0.5337
Epoch 6/100
284/284 [=====] - 11s 40ms/step - loss: 0.9841 - accuracy: 0.5026 - val_loss: 0.9695 - val_accuracy: 0.5062
Epoch 7/100
284/284 [=====] - 11s 39ms/step - loss: 0.9557 - accuracy: 0.5177 - val_loss: 0.9790 - val_accuracy: 0.5100
Epoch 8/100
284/284 [=====] - 11s 40ms/step - loss: 0.9425 - accuracy: 0.5274 - val_loss: 0.9130 - val_accuracy: 0.5524
Epoch 9/100
```

## Análisis del desempeño del modelo convolucional con regularización

### Función de pérdida (Loss)

- La pérdida del conjunto de entrenamiento alcanza valores de aproximadamente el 67.8%.
- El conjunto de validación muestra como resultado valores de aproximadamente el 66.8%.
- Este comportamiento indica que el dropout atenúa parcialmente el sobreajuste, aunque no logra eliminarlo por completo.



### Precisión (Accuracy)

- La precisión del conjunto de entrenamiento alcanza valores cercanos al 70 %, lo que refleja una ligeramente alta de ajuste al modelo.
- La precisión del conjunto de validación se estabiliza alrededor del 70 %, estando por debajo del desempeño del modelo convolucional sin dropout.
- La brecha entre entrenamiento y validación es menor, lo que sugiere una mejora en la generalización.



### Interpretación del comportamiento

- La incorporación de dropout reduce la dependencia excesiva de neuronas individuales durante el entrenamiento.
- El modelo mantiene una capacidad ligeramente alta de aprendizaje sin sacrificar significativamente el desempeño en datos no vistos.
- A pesar de la mejora, persisten indicios de sobreajuste, evidenciados por la diferencia entre las curvas de pérdida.

El modelo convolucional con dropout presenta un mejor equilibrio entre aprendizaje y generalización en comparación con las arquitecturas previas. Si bien no elimina completamente el sobreajuste, constituye una mejora significativa y demuestra la efectividad

de la regularización. Estos resultados respaldan la incorporación de técnicas adicionales, como el aumento de datos, para optimizar aún más el desempeño del modelo.

### **Implementación en una aplicación web**

El modelo se integra en una aplicación web que:

- Captura imágenes en tiempo real desde la cámara del dispositivo.
- Preprocesa cada imagen replicando exactamente los pasos del entrenamiento (redimensionamiento, normalización y forma)
- Realiza predicciones en tiempo real para clasificar si la imagen corresponde a un perro o un gato.

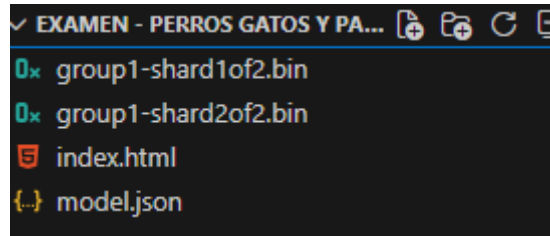
Para permitir el acceso a la cámara en dispositivos móviles, la aplicación se ejecuta en un entorno con HTTP, garantizando la compatibilidad con navegadores modernos.

### **Integración del modelo de aprendizaje automático en la aplicación web**

Una vez finalizado el proceso de entrenamiento y evaluación del modelo de aprendizaje automático en el entorno de Google Colab, se procede a su integración en una aplicación web con el objetivo de realizar predicciones en tiempo real. Para ello, el modelo entrenado es exportado al formato TensorFlow.js, el cual permite su ejecución directamente en el navegador sin necesidad de un servidor de procesamiento adicional.

El proceso de exportación genera un archivo principal denominado model.json, que contiene la arquitectura del modelo, así como uno o varios archivos con extensión .bin, los cuales almacenan los pesos aprendidos durante el entrenamiento. Estos archivos son indispensables

para el correcto funcionamiento del sistema y deben ubicarse en el mismo directorio que la aplicación web o en una ruta accesible desde esta.



Dentro de la aplicación web, el modelo es incorporado mediante la librería TensorFlow.js, la cual se carga desde un repositorio externo. Una vez cargada la librería, se utiliza una función específica para leer el archivo model.json llamada loadLayersModel, lo que permite reconstruir el modelo completo en memoria y asociar automáticamente los pesos correspondientes. Este paso representa el momento en el cual el modelo queda disponible para su uso dentro de la aplicación.

```
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@2.0.0/dist/tf.min.js"></script>
```

```
(async() => {  
  console.log("Cargando modelo...");  
  try {  
    modelo = await tf.loadLayersModel("model.json");  
    console.log("Modelo cargado");  
    statusMsg.innerText = "Modelo Cargado. Iniciando Cámara...";  
    statusMsg.style.color = "var(--neon-cyan)";  
    addLog("Modelo de 3 clases cargado exitosamente.", "log-system");  
  } catch (error) {  
    statusMsg.innerText = "Error cargando model.json";  
    statusMsg.style.color = "red";  
    console.error(error);  
  }  
})();
```



Debido a las restricciones de seguridad de los navegadores web, la aplicación no puede ejecutarse directamente desde el sistema de archivos local. Por esta razón, se requiere la utilización de un servidor web local que permita servir los archivos mediante el protocolo HTTP. Este procedimiento garantiza que el modelo pueda ser cargado correctamente y que la aplicación tenga acceso a los recursos necesarios.

```
PS C:\Users\ASUS\Documents\UNIVERSIDAD\6º SEMESTRE\IA\2P\EXAMEN - Perros gatos y pajaros> python -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

Una vez cargado el modelo, la aplicación web accede a la cámara del dispositivo para capturar imágenes en tiempo real. Cada imagen obtenida es procesada siguiendo exactamente los mismos pasos aplicados durante el entrenamiento del modelo, incluyendo el redimensionamiento, la normalización de los valores de los píxeles y la conversión a escala de grises. Posteriormente, la imagen es transformada en un tensor con la forma requerida por el modelo.

```
function mostrarCamara() {
    var opciones = { audio: false, video: { facingMode: facingMode, width: tamano, height: tamano } };
    if (navigator.mediaDevices.getUserMedia) {
        navigator.mediaDevices.getUserMedia(opciones)
            .then(function(stream) {
                currentStream = stream;
                video.srcObject = currentStream;
                procesarCamara();
                predecir();
                statusMsg.innerText = "SISTEMA ACTIVO";
            })
            .catch(function(err) {
                statusMsg.innerText = "Error de Cámara";
                statusMsg.style.color = "red";
                console.log(err);
            })
    } else {
        alert("No existe la funcion getUserMedia");
    }
}
```

```
function procesarCamara() {  
    if (!currentStream) return;  
  
    // Dibujamos el video en el canvas visible  
    ctx.drawImage(video, 0, 0, tamano, tamano, 0, 0, tamano, tamano);  
  
    // Loop de dibujo  
    setTimeout(procesarCamara, 20);  
}
```

Adicionalmente, para su uso en celulares, se creó la función cambiar cámara para utilizar tanto la cámara frontal como la cámara trasera.

```
function cambiarCamara() {  
    if (currentStream) {  
        currentStream.getTracks().forEach(track => track.stop());  
    }  
    facingMode = facingMode == "user" ? "environment" : "user";  
    canvas.style.transform = (facingMode == "user") ? "scaleX(-1)" : "scaleX(1)";  
    mostrarCamara();  
}
```

Dentro de la función de predicción, el modelo devuelve un array de tres probabilidades, es importante que el orden verificado en Google Colab, sea el mismo en el que se guarden las variables en el código de JavaScript.

```
Clases encontradas: ['bird', 'cat', 'dog']
```

```
// El resultado es un array de 3 probabilidades: [bird, cat, dog]
var probBird = resultado[0];
var probCat = resultado[1];
var probDog = resultado[2];

var pctBird = Math.round(probBird * 100);
var pctCat = Math.round(probCat * 100);
var pctDog = Math.round(probDog * 100);

scoreBird.innerText = pctBird + "%";
scoreCat.innerText = pctCat + "%";
scoreDog.innerText = pctDog + "%";

barBird.style.width = pctBird + "%";
barCat.style.width = pctCat + "%";
barDog.style.width = pctDog + "%";
```

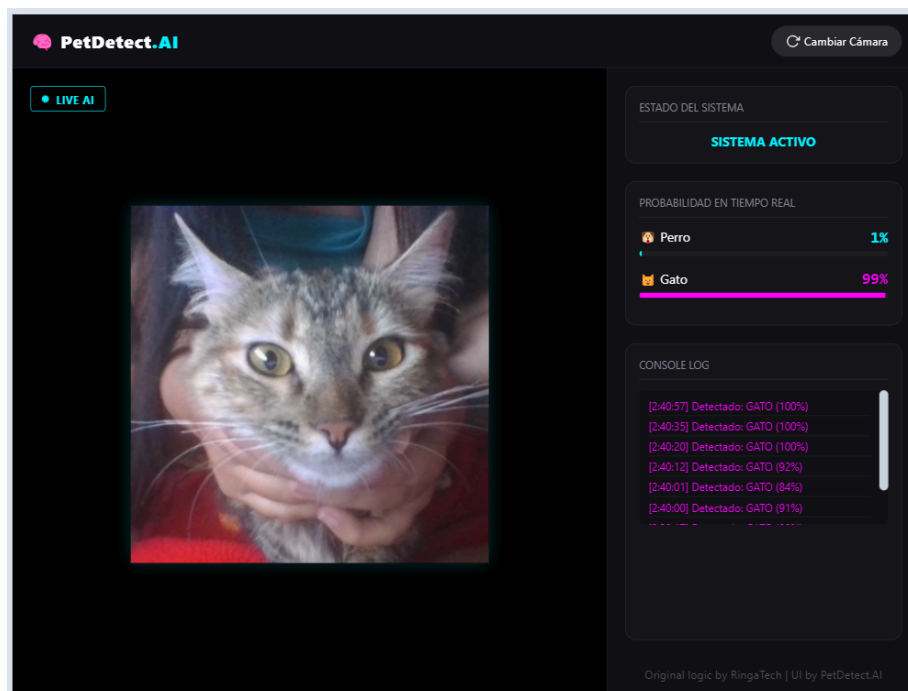
De esta manera, la integración del modelo en la aplicación web demuestra la viabilidad de utilizar modelos de aprendizaje automático entrenados en la nube para aplicaciones prácticas en tiempo real, facilitando la interacción entre el usuario y el sistema de reconocimiento de imágenes.

## Resultados

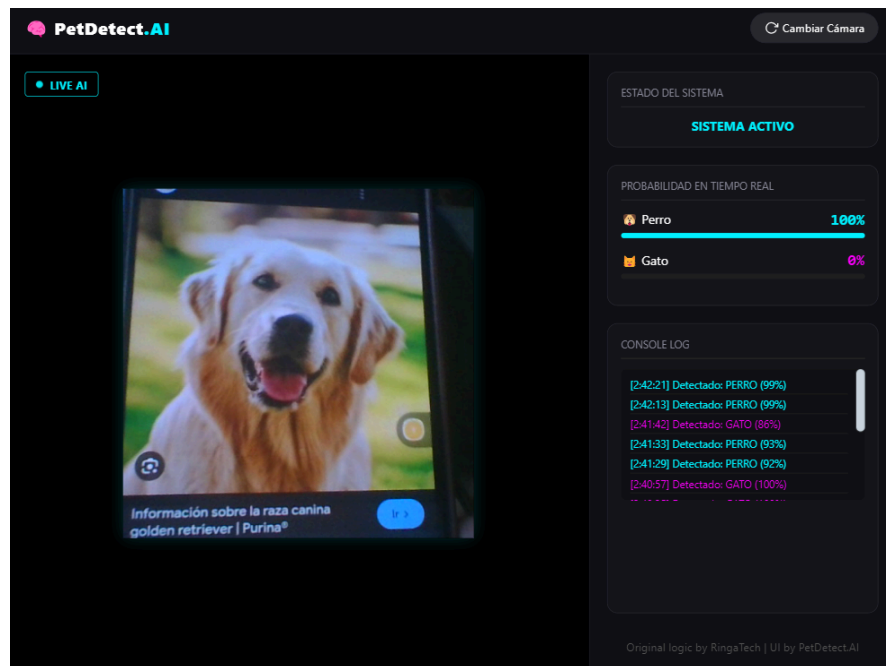
### Parte 1

Los resultados obtenidos demuestran el correcto funcionamiento del sistema de reconocimiento de imágenes desarrollado, tanto a nivel del modelo de aprendizaje automático como de su integración en la aplicación web. Durante las pruebas realizadas, el sistema fue capaz de identificar en tiempo real imágenes correspondientes a perros y gatos utilizando la cámara del dispositivo, mostrando de manera inmediata la predicción y el nivel de confianza asociado a cada clase.

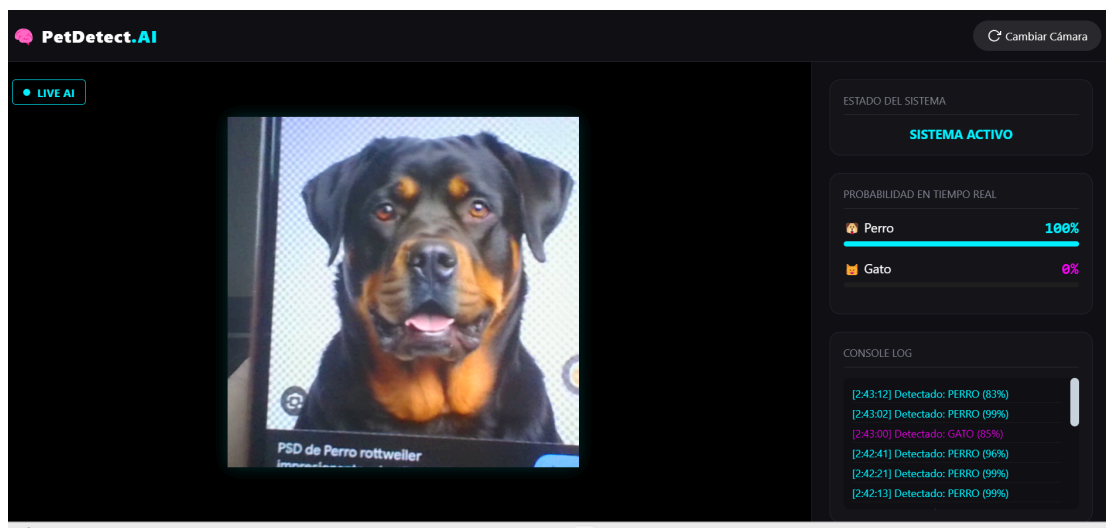
En la primera prueba, al presentar una imagen correspondiente a un gato, el sistema clasificó correctamente la imagen como gato, alcanzando probabilidades superiores al 99 %, mientras que la probabilidad asignada a la clase perro fue mínima. Este comportamiento se mantuvo estable durante varios segundos, lo cual indica consistencia en la predicción y un bajo nivel de fluctuación en los resultados.



Posteriormente, al evaluar imágenes correspondientes a diferentes razas de perros, el sistema identificó correctamente la clase perro, alcanzando valores de probabilidad cercanos al 100 %. En estas pruebas, la clase gato presentó probabilidades cercanas a cero, lo que evidencia una clara separación entre ambas clases por parte del modelo entrenado.



Adicionalmente, se observó que el sistema responde adecuadamente ante variaciones en la imagen de entrada, tales como cambios en la iluminación, calidad de la imagen y uso de fotografías mostradas desde otros dispositivos. A pesar de estas variaciones, el modelo mantuvo un alto nivel de precisión, lo que sugiere una adecuada capacidad de generalización.



La aplicación web mostró los resultados de manera clara y comprensible para el usuario, indicando el estado del sistema como activo, las probabilidades de cada clase mediante

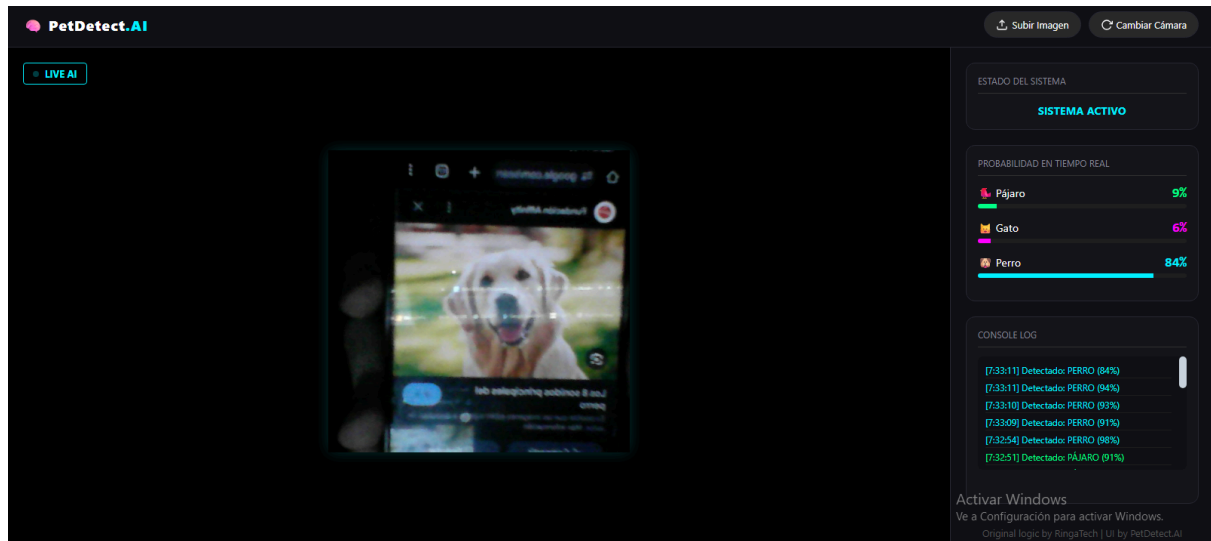
porcentajes y barras visuales, y un registro temporal de detecciones en la consola. Esta retroalimentación visual permitió verificar en tiempo real el comportamiento del modelo y confirmar la correcta integración entre el procesamiento de imágenes, la predicción del modelo y la interfaz gráfica. En conjunto, los resultados obtenidos validan que el modelo de aprendizaje automático implementado es capaz de realizar clasificaciones precisas en tiempo real, y que su integración en una aplicación web funcional representa una solución viable para sistemas de reconocimiento de imágenes basados en redes neuronales convolucionales.

## Parte 2

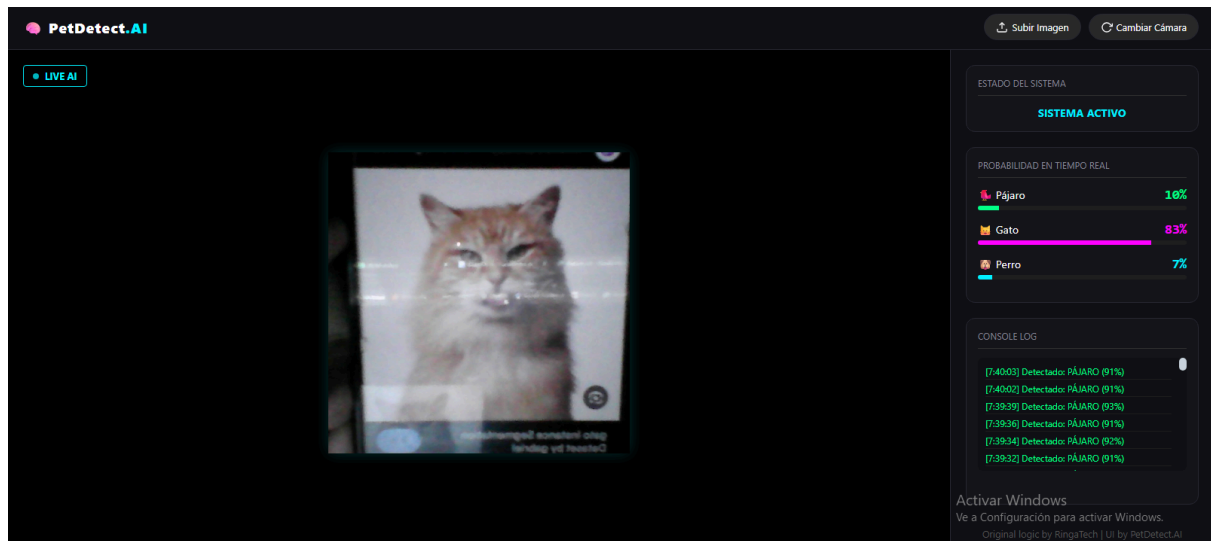
La conexión con la interfaz gráfica y el modelo se realizó de la misma manera que en la parte uno, pero con la diferencia de que en el array de probabilidades que se obtiene, es importante configurar el mismo orden de los datos que se tiene en Google Colab. En este caso, el orden es pájaros, gatos y perros.

```
// El resultado es un array de 3 probabilidades: [bird, cat, dog]
var probBird = resultado[0];
var probCat = resultado[1];
var probDog = resultado[2];
```

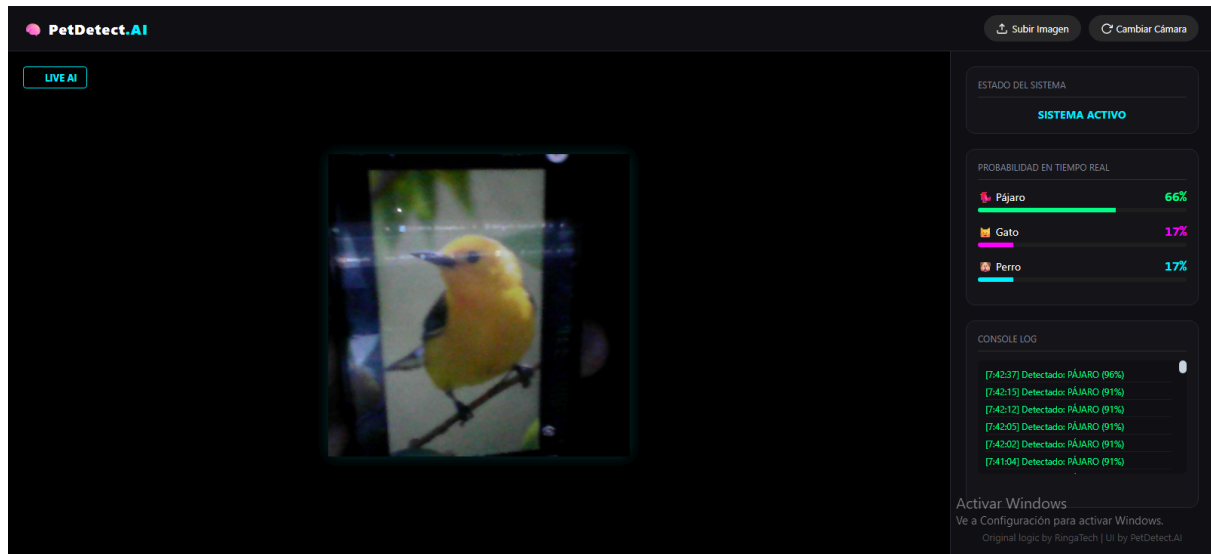
En cuanto al sistema de detección con los pájaros, se utilizó el mismo método de conexión y la misma interfaz gráfica, sin embargo, se muestran las barras de porcentajes de los tres animales.



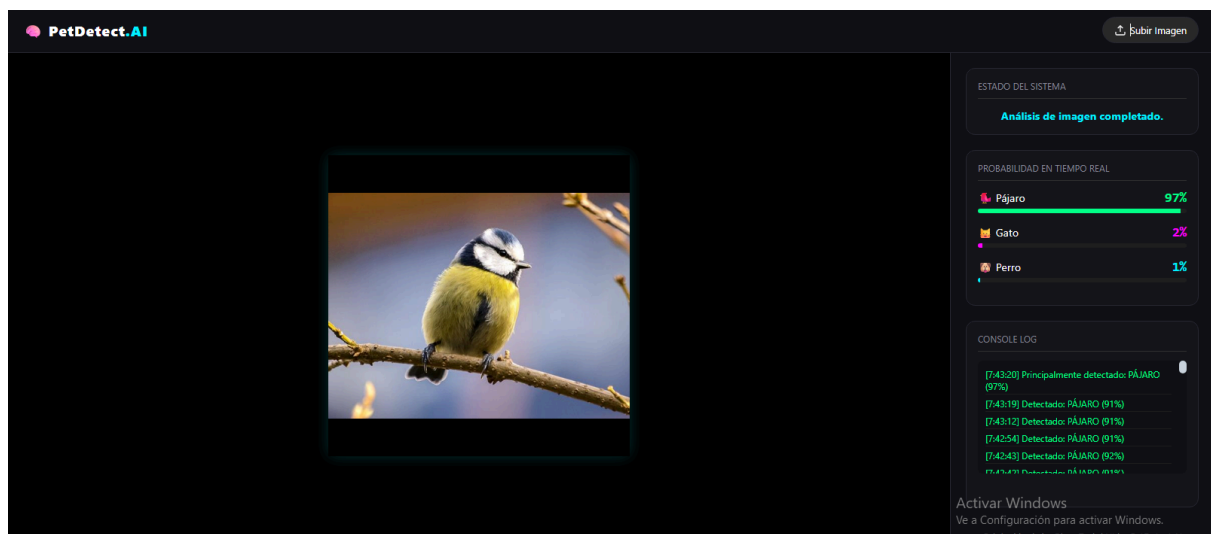
De igual manera, el sistema sigue detectando las características de un gato, evidenciando que no perdió esa funcionalidad.



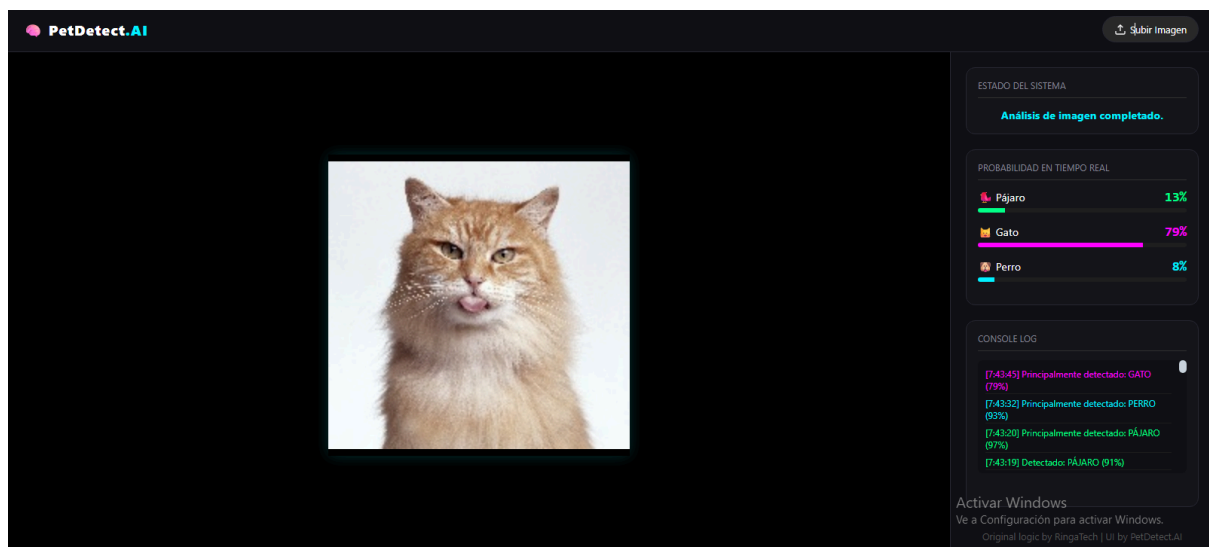
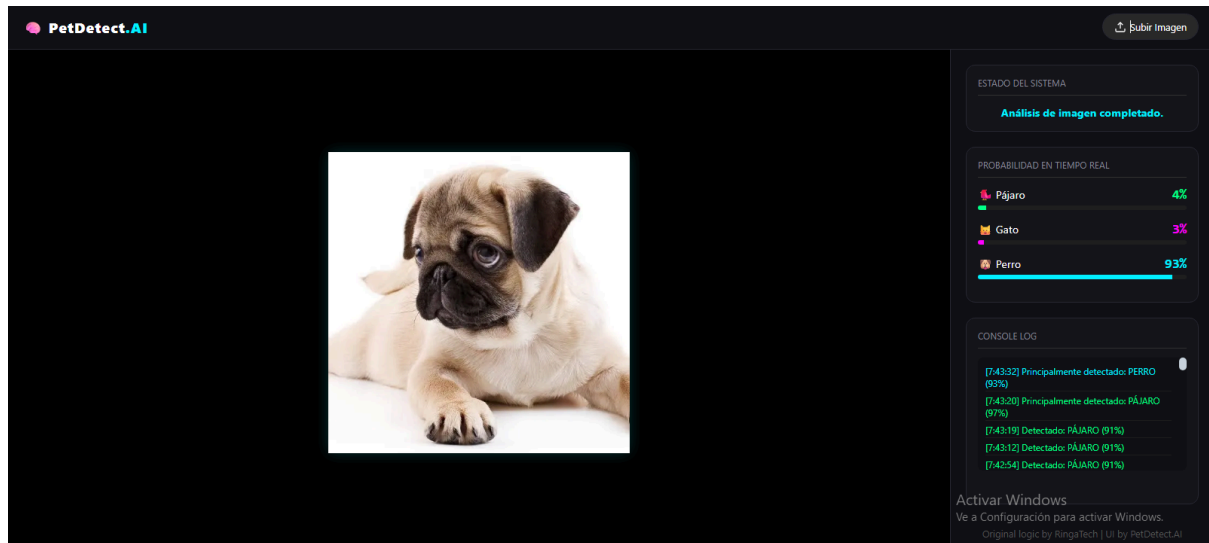
Ahora, el añadido en esta sección es el animal pájaro, como se puede evidenciar, el sistema lo sigue detectando perfectamente.



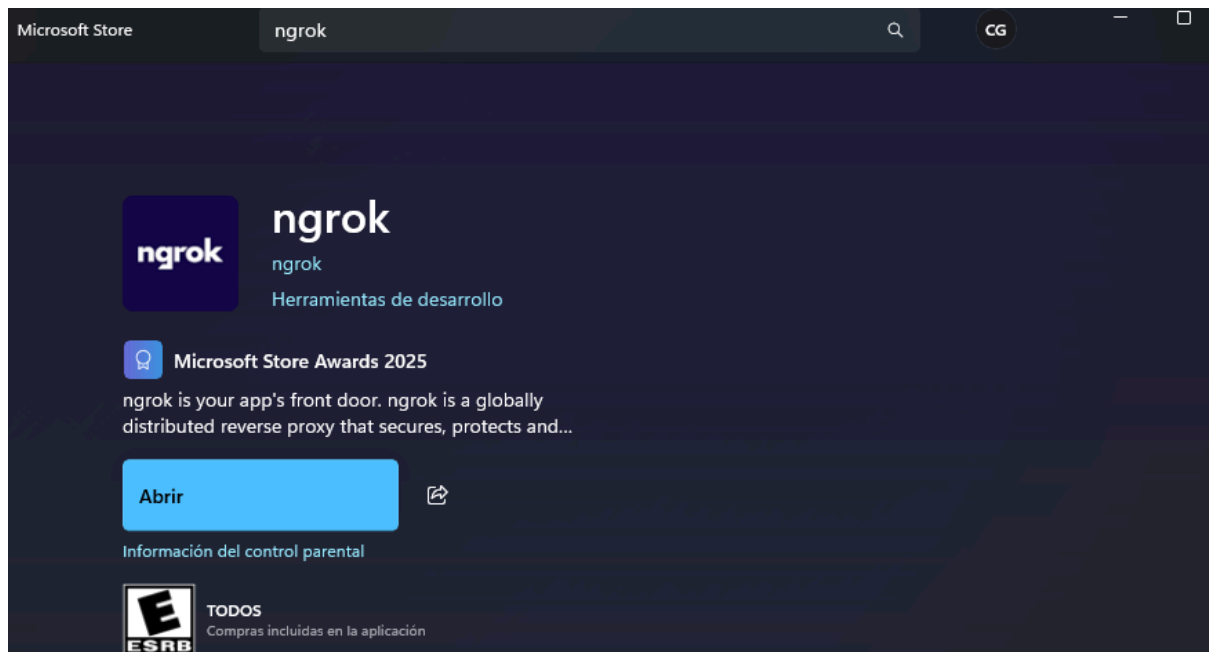
Adicionalmente, en la parte 2 se configuró la funcionalidad de subir una imagen y que el sistema lo detecte, devolviendo un porcentaje fijo para la imagen.



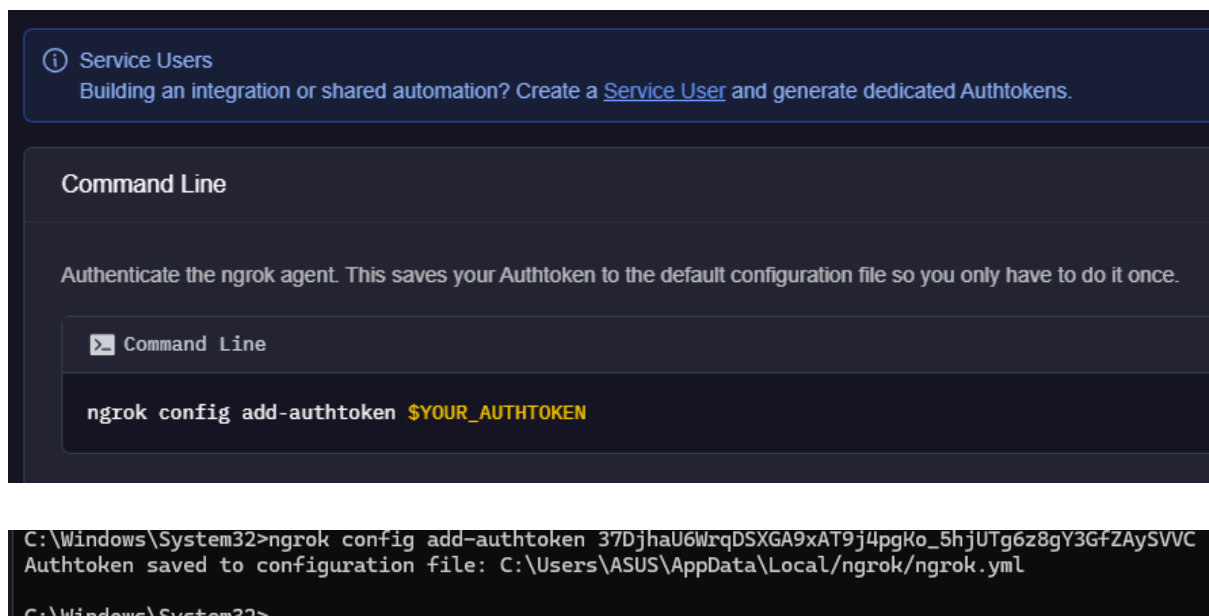




Adicionalmente para el funcionamiento del aplicativo en versión móvil, se configuró un servidor de ngrok. El primer paso es descargar la aplicación de ngrok desde la Microsoft Store.



Continuamente, en la página de ngrok, se habilitó la API Key y se ejecutó el comando respectivo en la terminal de ngrok descargada previamente.



Finalmente, una vez que el servidor de python este levantado, se debe apuntar a dicho servidor desde la terminal de ngrok.

```
PS C:\Users\ASUS\Documents\UNIVERSIDAD\6º SEMESTRE\IA\2P\EXAMEN - Perros gatos y pajaros> python -m http.server 8000

::1 - - [22/Dec/2025 18:11:40] code 404, message File not found
::1 - - [22/Dec/2025 18:11:40] "GET /favicon.ico HTTP/1.1" 404 -
::1 - - [22/Dec/2025 18:11:47] "GET / HTTP/1.1" 200 -
::1 - - [22/Dec/2025 18:11:57] "GET / HTTP/1.1" 200 -
::1 - - [22/Dec/2025 18:12:20] "GET / HTTP/1.1" 200 -
::1 - - [22/Dec/2025 18:14:56] "GET / HTTP/1.1" 200 -
::1 - - [22/Dec/2025 18:20:13] "GET / HTTP/1.1" 304 -
::1 - - [22/Dec/2025 18:20:14] code 404, message File not found
::1 - - [22/Dec/2025 18:20:14] "GET /favicon.ico HTTP/1.1" 404 -

```

```
ngrok (Ctrl+C to

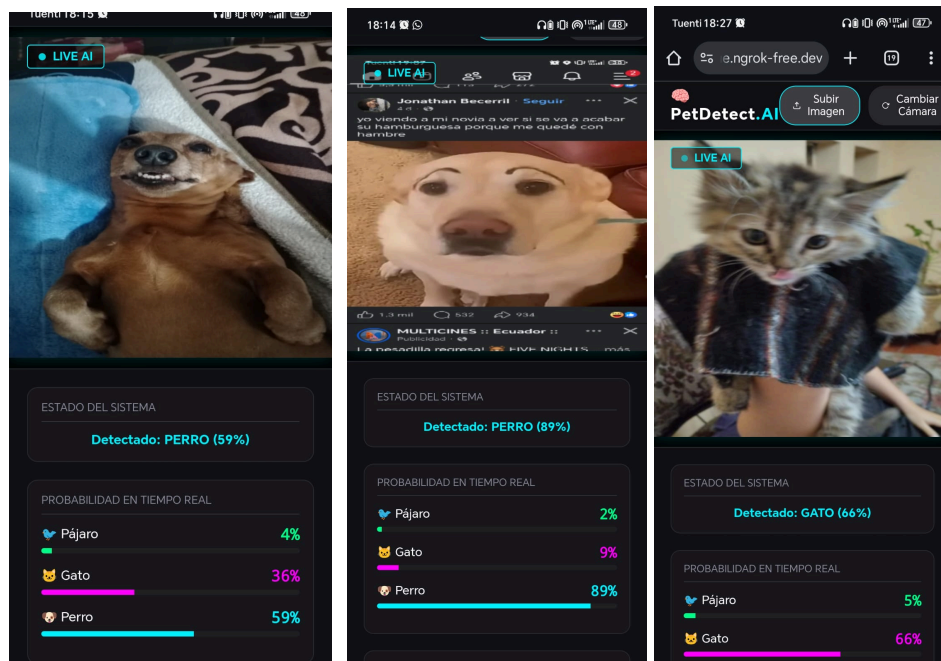
♦ One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             cagonzalez12@espe.edu.ec (Plan: Free)
Update              update available (version 3.34.1, Ctrl-U to update)
Version             3.24.0-msix
Region              United States (us)
Latency             313ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://jonas-semimountainous-monique.ngrok-free.dev -> http://localhost:8000

Connections          ttl    opn    rt1    rt5    p50    p90
                   16     0     0.00   0.01   0.03   7.85

HTTP Requests
-----
18:20:14.550 -14 GET /favicon.ico          404 File not found
18:20:13.855 -13 GET /                    304 Not Modified
18:14:56.191 -56 GET /                    200 OK
18:12:20.950 -20 GET /                    200 OK
18:11:57.302 -57 GET /                    200 OK
18:11:47.825 -47 GET /                    200 OK
18:11:40.904 -40 GET /favicon.ico          404 File not found
18:11:26.361 -26 GET /                    200 OK
18:11:19.838 -19 GET /                    502 Bad Gateway
18:09:46.276 -46 GET /                    304 Not Modified

```



## Conclusiones

A partir del análisis y preparación del conjunto de datos de imágenes de perros y gatos, se logró establecer una base sólida para el entrenamiento de un modelo de aprendizaje supervisado, asegurando la correcta correspondencia entre imágenes y etiquetas. Este proceso permitió trabajar con datos consistentes y adecuados para la tarea de clasificación.

La aplicación de técnicas de preprocesamiento, como el redimensionamiento, la normalización y la conversión a escala de grises, resultó fundamental para optimizar el entrenamiento del modelo. Estas técnicas contribuyeron a reducir la complejidad de los datos, mejorar la estabilidad del proceso de aprendizaje y facilitar la convergencia de los modelos entrenados.

El diseño y entrenamiento de diferentes arquitecturas de redes neuronales permitió comparar el comportamiento de modelos densos y redes neuronales convolucionales. A partir de esta comparación, se evidenció que las redes convolucionales son más adecuadas para el reconocimiento de imágenes, debido a su capacidad para extraer características espaciales relevantes.

La evaluación del desempeño de los modelos mediante métricas de precisión y pérdida permitió identificar problemas como el sobreajuste, especialmente en modelos más simples o entrenados sin técnicas adicionales. Este análisis fue clave para comprender las limitaciones de cada arquitectura y tomar decisiones informadas sobre la selección del modelo final.

La implementación de técnicas de aumento de datos demostró ser efectiva para mejorar la capacidad de generalización del modelo, reduciendo el sobreajuste y logrando un mejor equilibrio entre los resultados de entrenamiento y validación. Gracias a esto, se obtuvo un modelo más robusto y confiable ante datos no vistos.

La exportación del modelo entrenado y su integración en una aplicación web permitió comprobar la viabilidad de utilizar modelos de aprendizaje automático en entornos interactivos, logrando realizar predicciones en tiempo real a partir de la cámara del dispositivo y mostrando los resultados de manera clara al usuario.

Finalmente, la incorporación de un tercer tipo de animal mediante un nuevo conjunto de datos permitió ampliar el alcance del sistema hacia un problema de clasificación multiclase. El análisis del impacto de esta nueva clase evidenció un aumento en la complejidad del modelo, así como la necesidad de un mayor cuidado en el entrenamiento y la evaluación, confirmando la importancia de contar con datos balanceados y técnicas adecuadas para mantener un buen rendimiento del sistema.

### **Recomendaciones**

Se recomienda ampliar y diversificar el conjunto de datos utilizado, incorporando imágenes con distintas condiciones de iluminación, fondos y ángulos, con el fin de mejorar la capacidad de generalización del modelo y reducir posibles sesgos en la clasificación.

Es aconsejable mantener un balance adecuado entre las clases del conjunto de datos, especialmente al incorporar nuevos tipos de animales, para evitar que el modelo favorezca una clase sobre las demás y asegurar un rendimiento más estable.

Se sugiere evaluar arquitecturas de redes neuronales más profundas o modelos preentrenados mediante técnicas de transfer learning, lo cual podría mejorar la precisión del sistema sin incrementar significativamente el tiempo de entrenamiento.

Se recomienda continuar utilizando técnicas de aumento de datos y explorar métodos adicionales de regularización, como batch normalization o ajustes en la tasa de aprendizaje, para minimizar el sobreajuste durante el entrenamiento.

Para futuras implementaciones, se sugiere optimizar el desempeño de la aplicación web, evaluando distintos backends de TensorFlow.js y considerando la ejecución del modelo en dispositivos con capacidades de hardware variadas.

Finalmente, se recomienda realizar pruebas del sistema en diferentes entornos y dispositivos, con el fin de validar su robustez y confiabilidad en escenarios reales de uso.

## Referencias

IBM. (2021, October 6). *redes neuronales convolucionales*. Ibm.com.  
<https://www.ibm.com/es-es/think/topics/convolutional-neural-networks>

Alicante, V. (2025, April 9). *¿Qué es una red neuronal artificial y cómo funcionan?*  
Universidad Europea; Universidad Europea | Universidad presencial (Madrid, Valencia, Alicante, Canarias, Málaga) y Online.  
<https://universidadeuropea.com/blog/redes-neuronales-artificiales/>

Belcic, I., & Stryker, C. (2025, September 12). *aprendizaje supervisado*. Ibm.com.  
<https://www.ibm.com/es-es/think/topics/supervised-learning>

*¿Qué es el aprendizaje automático? Tipos y usos*. (2025). Google Cloud.  
<https://cloud.google.com/learn/what-is-machine-learning?hl=es-419>